



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

WYDZIAŁ INŻYNIERII METALI I INFORMATYKI PRZEMYSŁOWEJ

KATEDRA INFORMATYKI STOSOWANEJ I MODELOWANIA

Projekt dyplomowy

Wykorzystanie technik DevOps w implementacji automatycznego procesu CI/CD do uczenia i wersjonowania modeli sztucznej inteligencji: analiza utrzymania systemów wykorzystujących sztuczną inteligencję w porównaniu do metod manualnych

Using DevOps techniques to implement an automated CI/CD process for training and versioning AI models: an analysis of the maintenance of AI-based systems compared to manual methods

Autor: Rafał Zabłotni
Kierunek studiów: Inżynieria Obliczeniowa
Opiekun pracy: mgr inż. Piotr Hajder

Kraków, 2026

Spis treści

1	Wstęp	3
1.1	Geneza	3
1.2	Założenia	4
2	Wprowadzenie Teoretyczne	5
2.1	DevOps	5
2.2	Ciągła Integracja i Ciągłe Dostarczanie (CI/CD)	5
2.3	MLOps	6
2.4	Kontrola wersji — Git i GitHub	6
2.5	GitHub Actions	7
2.6	Konteneryzacja — Docker	7
2.7	Kubernetes i k3s	8
2.8	Infrastruktura jako kod — Terraform i Helm	8
2.9	Wersjonowanie danych — DVC	8
2.10	Śledzenie eksperymentów — MLflow	9
2.11	Monitoring — Prometheus i Grafana	9
3	Architektura Systemu	10
3.1	Cele i ograniczenia projektowe	10
3.2	Struktura repozytoriów	10
3.3	Warstwy architektury	11
3.3.1	Warstwa użytkownika	12
3.3.2	Warstwa kontroli wersji i automatyzacji	12
3.3.3	Warstwa infrastruktury chmurowej	13
3.3.4	Warstwa platformy ML i serwowania	13
3.4	Kluczowe decyzje projektowe	13
4	Model Sztucznej Inteligencji	15
4.1	Charakterystyka problemu segmentacji semantycznej	15
4.2	Architektura modelu	15
4.3	Dane treningowe i przygotowanie zbioru danych	16
4.4	Proces uczenia i metryki jakości	16
4.5	Model jako artefakt w procesie CI/CD	17
5	Pipeline CI/CD	18
5.1	Hook pre-push	18
5.2	Struktura workflow GitHub Actions	19
5.3	Walidacja danych	20
5.4	Pipeline treningowy	21
5.5	Bramka jakości	23
6	Wdrożenie Systemu	25

6.1	Infrastruktura Terraform	25
6.2	Konfiguracja EC2 i k3s	27
6.3	Dostępne porty i usługi	29
6.4	Konfiguracja Helm	30
6.5	Stos monitorowania	32
6.6	Workflow użytkownika	33
6.6.1	Scenariusz 1: Wdrożenie przez automatyczny pipeline	33
6.6.2	Scenariusz 2: Ręczne wdrożenie i korzystanie z API (zalecany)	33
6.6.3	Scenariusz 3: Lokalna predykcja (wyłącznie dla deweloperów)	34
6.7	Dostępne operacje zarządzania wdrożeniem	35
7	Analiza Wyników i Działania Systemu	36
7.1	Metodologia porównania	36
7.2	Czasy wykonania scenariuszy	36
7.3	Metryki modelu	36
7.4	Koszty infrastruktury AWS	41
7.5	Ocena jakościowa	42
7.5.1	Łatwość wdrożenia	42
7.5.2	Stabilność systemu	42
7.5.3	Skalowalność	42
8	Podsumowanie i wnioski	43
8.1	Efektywność automatyzacji	43
8.2	Porównanie mocnych i słabych stron obu podejść	43
8.3	Obszary optymalizacji	44
8.4	Podsumowanie	44
8.5	Wykorzystanie narzędzi sztucznej inteligencji	45
	Literatura	46

1 Wstęp

1.1 Geneza

Pomysł na realizację niniejszego projektu zrodził się podczas wykonywania codziennych obowiązków zawodowych. W trakcie pracy nad jednym z projektów zaobserwowano interesującą tendencję związaną z procesem tworzenia testów modułowych. Znaczna część kodu testowego okazywała się redundantna, a wiele mechanizmów było wielokrotnie powielanych poprzez kopiowanie oraz wprowadzanie drobnych modyfikacji do istniejących fragmentów kodu.

Naturalnym rozwiązaniem tego problemu mogłoby wydawać się wydzielanie powtarzalnych elementów do funkcji pomocniczych lub bibliotek testowych oraz utrzymywanie ich w uporządkowanej i współdzielonej formie. Podejście to jednak wiąże się z istotnymi trudnościami, zarówno natury organizacyjnej, jak i technicznej. W praktyce często prowadzi ono do nadmiernego rozrostu bibliotek pomocniczych, niejednoznacznych interfejsów oraz problemów z ich długoterminowym utrzymaniem.

W szczególności pojawiają się następujące problemy:

1. Jak określić moment, w którym dana funkcjonalność jest wystarczająco generyczna, aby mogła zostać wydzielona do wspólnej biblioteki?
2. Jak zminimalizować ryzyko tworzenia funkcji lokalnie w plikach testowych, które z czasem zaczynają być wykorzystywane przez inne testy, prowadząc do niejawnych zależności pomiędzy teoretycznie niezależnymi modułami?
3. Jak zapewnić spójny sposób tworzenia oraz utrzymania takich funkcji w projektach rozwijanych przez dziesiątki, a niekiedy setki programistów, o różnym poziomie doświadczenia i odmiennych nawykach pracy?

Jednocześnie, jak powszechnie wiadomo, modele sztucznej inteligencji uczą się na podstawie zbiorów danych treningowych, a następnie wykorzystują wyuczone wzorce do rozwiązywania nowych problemów. Obserwacja ta stała się punktem wyjścia do rozważenia możliwości zastosowania modeli AI w obszarze automatycznego generowania wspólnych, powtarzalnych fragmentów kodu testowego na podstawie już istniejących przykładów.

Jednakże, aby taki system mógł być skutecznie wykorzystany w praktyce, konieczne jest zapewnienie odpowiedniego procesu ciągłej integracji i dostarczania (CI/CD), który umożliwi automatyczne trenowanie, ocenę jakości oraz wdrażanie modeli AI w sposób powtarzalny i kontrolowany, lub oddelegowanie pracownika do utrzymania takiego procesu w sposób manualny.

W ten sposób pojawiają dwa kluczowe pytania, które stanowią oś projektu:

1. Czy możliwe i opłacalne jest stworzenie modelu SI który będzie obsługiwał automatyczne generowanie fragmentów kodu testowego?
2. Czy bardziej opłacalne jest oddelegowanie jednego pracownika do ręcznego utrzymywania i rozwijania tego oprogramowania, czy też lepszym rozwiązaniem jest automatyzacja tego procesu z wykorzystaniem narzędzi DevOps oraz pipeline'ów CI/CD?

1.2 Założenia

Ze względu na złożoność oryginalnego problemu generowania kodu testowego oraz ograniczony czas realizacji projektu, zdecydowano się na zastosowanie problemu zastępczego, który pozwala w pełni zweryfikować hipotezę bez konieczności budowania zaawansowanego modelu językowego. Jako taki substytut wybrano segmentację semantyczną obrazów wodomierzy z wykorzystaniem architektury U-Net. Wybór ten jest celowy: model segmentacji obrazów posiada wyraźnie zdefiniowane metryki jakości (współczynnik Dice, IoU), umiarkowane wymagania obliczeniowe umożliwiające trening w ramach dostępnego budżetu, a dostępność wstępnie wytrenowanego modelu bazowego pozwala na obiektywne porównanie kolejnych wersji. Dla uproszczenia przyjęto, że model SI dostarczany jest przez zewnętrzną firmę, natomiast zakres pracy obejmuje jego dalsze uczenie, ocenę jakości oraz wdrażanie. Takie założenie odzwierciedla realny scenariusz w środowisku produkcyjnym, gdzie zespół DevOps otrzymuje gotowy model i odpowiada za zarządzanie jego cyklem życia.

W tak zdefiniowanym kontekście drugie pytanie sformułowane w poprzedniej sekcji pozostaje w pełni aktualne: czy bardziej opłacalne jest oddelegowanie pracownika do ręcznego utrzymywania i rozwijania modelu, czy też lepszym rozwiązaniem jest automatyzacja tego procesu z wykorzystaniem narzędzi DevOps i pipeline'ów CI/CD? W podejściu manualnym proces ponownego uczenia i wdrażania modelu wymaga ręcznego przygotowania danych, uruchamiania treningu, walidacji wyników oraz publikacji nowej wersji. Takie podejście, choć proste koncepcyjnie, wiąże się z ryzykiem błędów ludzkich, trudnościami w zachowaniu powtarzalności eksperymentów oraz ograniczoną skalowalnością. Automatyzacja natomiast umożliwia jednoznaczne powiązanie wersji modelu z wersją danych treningowych, wprowadzenie obiektywnej bramki jakości i eliminację ręcznych kroków, które mogą prowadzić do niespójności.

W ramach niniejszej pracy opracowano kompletny, zautomatyzowany system CI/CD dla modeli sztucznej inteligencji. System obejmuje wersjonowanie danych treningowych (DVC), śledzenie eksperymentów i rejestr modeli (MLflow), automatyczny pipeline treningowy uruchamiany przez GitHub Actions, bramkę jakości porównującą nowy model z aktualną wersją produkcyjną, a także wdrożenie modelu w środowisku chmurowym (k3s na EC2) z możliwością użycia aplikacji oraz monitoringiem (Prometheus, Grafana). Całość zaprojektowano tak, aby dodanie nowych danych treningowych przez użytkownika wymagało jedynie wykonania `git push`, a dalszy proces odbywał się bez jego udziału. Tak zdefiniowany zakres pozwala na bezpośrednie porównanie kosztów i nakładu pracy podejścia manualnego i automatycznego, co stanowi główny cel analityczny pracy.

2 Wprowadzenie Teoretyczne

Realizacja projektu opisanego w niniejszej pracy opiera się na zestawie technologii i koncepcji z obszaru inżynierii oprogramowania, uczenia maszynowego i infrastruktury chmurowej. Niniejszy rozdział przedstawia teoretyczne podstawy każdego z tych elementów, stanowiąc punkt odniesienia dla decyzji projektowych opisanych w kolejnych rozdziałach. Omówiono kolejno: filozofię DevOps i mechanizmy CI/CD, podejście MLOps jako ich rozszerzenie na świat modeli ML, a następnie konkretne narzędzia zastosowane w projekcie: Git i GitHub, GitHub Actions, Docker, Kubernetes (k3s), Terraform z Helm, MLflow, DVC oraz Prometheus z Grafaną.

2.1 DevOps

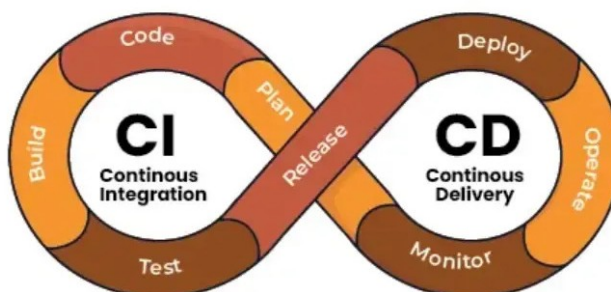
DevOps to zbiór praktyk, kultury organizacyjnej i narzędzi, których celem jest skrócenie cyklu życia oprogramowania przy jednoczesnym zapewnieniu wysokiej jakości dostarczanych produktów. Termin pochodzi od połączenia słów *Development* i *Operations*, co odzwierciedla dążenie do przełamania tradycyjnej bariery między zespołami deweloperskimi a operacyjnymi.

Kluczowym elementem filozofii DevOps jest pętla ciągłego doskonalenia, obejmująca planowanie, kodowanie, budowanie, testowanie, wdrażanie, uruchamianie oraz monitorowanie. Każdy obrót pętli dostarcza informacje zwrotne, które pozwalają szybciej reagować na błędy i wymagania użytkowników. W kontekście niniejszej pracy DevOps stanowi ramy koncepcyjne, w których osadzony jest cały projekt - automatyzacja procesu trenowania i wdrażania modelu SI jest bezpośrednią aplikacją tych zasad [13].

2.2 Ciągła Integracja i Ciągłe Dostarczanie (CI/CD)

Ciągła Integracja (ang. *Continuous Integration*, CI) polega na automatycznym scalaniu i weryfikacji zmian kodu wielokrotnie w ciągu dnia. Każda zmiana uruchamia zdefiniowany zestaw testów, co pozwala wykryć błędy integracyjne natychmiast po ich wprowadzeniu, zamiast odkrywać je dopiero na etapie wydania.

Ciągłe Dostarczanie (ang. *Continuous Delivery*) rozszerza CI o automatyczne przygotowanie artefaktów gotowych do wdrożenia. Ciągłe Wdrażanie (ang. *Continuous Deployment*) idzie krok dalej - każda zmiana spełniająca kryteria jakości jest wdrażana do produkcji.



Rys. 1 Przykład flow CI/CD [16]

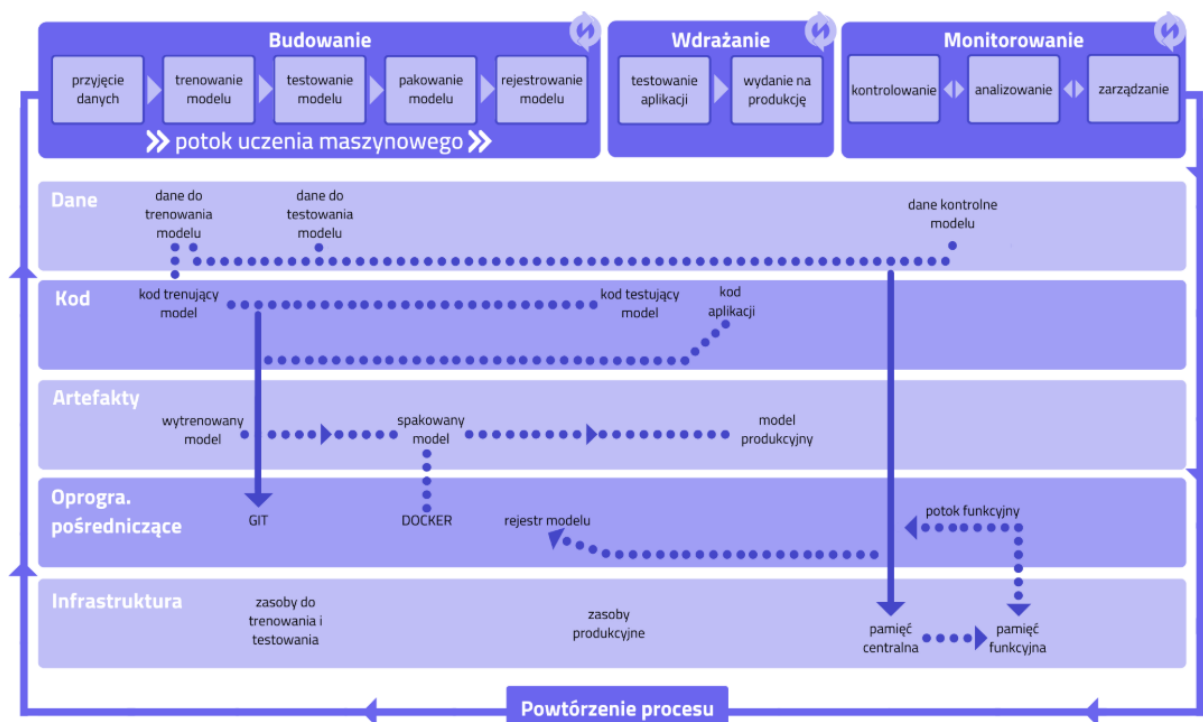
Typowy pipeline CI/CD składa się z następujących etapów:

1. weryfikacja kodu (linting, testy jednostkowe),
2. budowanie artefaktów (np. obrazu Docker),
3. testy integracyjne i akceptacyjne,
4. wdrożenie na środowisko docelowe,
5. weryfikacja po wdrożeniu (smoke tests).

2.3 MLOps

MLOps (ang. *Machine Learning Operations*) to dyscyplina stosująca zasady DevOps do specyfikacji cyklu życia modeli uczenia maszynowego. W odróżnieniu od klasycznego oprogramowania, gdzie artefaktem jest kod, w ML artefaktem jest model — wynik trenowania na konkretnym zbiorze danych. To wprowadza kilka dodatkowych wyzwań:

- dane treningowe zmieniają się niezależnie od kodu,
- wyniki trenowania są niedeterministyczne,
- jakość modelu musi być mierzona obiektywnie przed każdym wdrożeniem,
- modele degradują się w czasie (tzw. *data drift*).



Rys. 2 Przykład flow CI/CD [5]

MLflow spina cały cykl życia modelu ML — od rejestrowania eksperymentów, parametrów i metryk podczas treningu, przez wersjonowanie oraz rejestrację modeli w Model Registry, aż po zarządzanie wdrożeniem i monitorowanie kolejnych wersji w środowisku produkcyjnym.

2.4 Kontrola wersji — Git i GitHub

Git to rozproszony system kontroli wersji, będący de facto standardem w nowoczesnym tworzeniu oprogramowania [3]. Przechowuje pełną historię zmian w postaci grafu migawek (ang. *snapshots*), umożliwiając równoległą pracę na gałęziach (ang. *branches*), scalanie zmian oraz

cofanie się do dowolnego punktu w historii projektu.

Kluczowe koncepcje Gita:

- **Commit** — migawka stanu repozytorium w danym momencie wraz z metadanymi (autor, czas, wiadomość),
- **Branch** — niezależna linia rozwoju; umożliwia pracę nad funkcjonalnością bez wpływu na gałąź główną,
- **Merge / Pull Request** — scalenie zmian z jednej gałęzi do drugiej, zazwyczaj poprzedzone przeglądem kodu,
- **Hook** — skrypt wywoływany automatycznie przez zdarzenie Git (np. `pre-push`).

GitHub to platforma hostingowa dla repozytoriów Git, rozbudowana o funkcje współpracy zespołowej: pull requesty, przeglądy kodu, zarządzanie zadaniami oraz zintegrowane CI/CD w postaci GitHub Actions. W kontekście niniejszego projektu GitHub pełni rolę centralnego punktu integracji. Każda zmiana przechodzi przez repozytorium i może automatycznie uruchamiać kolejne etapy pipeline'u.

2.5 GitHub Actions

GitHub Actions to platforma automatyzacji wbudowana w GitHub, pozwalająca definiować przepływy pracy (ang. *workflows*) jako pliki YAML przechowywane bezpośrednio w repozytorium [6]. Workflow uruchamiane są w odpowiedzi na zdarzenia: push, pull request, harmonogram (cron) lub wywołanie ręczne.

Podstawowe pojęcia:

- **Workflow** — zestaw automatycznych zadań zdefiniowany w pliku `.github/workflows/*.yaml`,
- **Job** — jednostka wykonania złożona z kroków (steps); domyślnie uruchamiana równolegle z innymi jobami,
- **Step** — pojedyncza operacja w ramach joba: wywołanie skryptu lub gotowej akcji,
- **Runner** — maszyna wykonująca joba; może być zarządzana przez GitHub (hosted) lub przez użytkownika (self-hosted),
- **Artefakt** — plik lub katalog przekazywany między jobami albo pobierany po zakończeniu workflow.

Kluczową cechą GitHub Actions jest możliwość użycia **self-hosted runners**. Maszyny zarządzane przez użytkownika, co w niniejszym projekcie pozwala uruchamiać trening bezpośrednio na instancji EC2 wyposażonej w odpowiedni sprzęt obliczeniowy, bez potrzeby przekazywania danych treningowych przez sieć do zewnętrznego runnera.

2.6 Konteneryzacja — Docker

Docker to platforma do tworzenia, dystrybucji i uruchamiania aplikacji w kontenerach [14]. Kontener to izolowane środowisko uruchomieniowe zawierające aplikację wraz ze wszystkimi jej zależnościami: bibliotekami, interpreterem, zmiennymi środowiskowymi. Dzięki temu aplikacja działa identycznie na maszynie dewelopera, serwerze testowym i produkcji, eliminując problem „u mnie działa”.

Kluczowe pojęcia:

- **Obraz (image)** — niezmienny szablon środowiska definiowany plikiem `Dockerfile`,
- **Kontener** — działająca instancja obrazu; izolowana od systemu hosta i innych kontenerów,
- **Rejestr (registry)** — repozytorium obrazów (np. Docker Hub, Amazon ECR).

W projekcie Docker służy do pakowania API modelu oraz do zapewnienia powtarzalności środowiska treningowego. Obraz z modelem jest budowany automatycznie w pipeline po zatwierdzeniu nowej wersji modelu.

2.7 Kubernetes i k3s

Kubernetes (K8s) to system do orkiestracji kontenerów, który automatyzuje ich wdrażanie, skalowanie i zarządzanie [2]. Zamiast uruchamiać kontenery ręcznie na konkretnych maszynach, Kubernetes przyjmuje deklaracyjny opis pożądanego stanu (ile replik, jakie zasoby, jak reagować na awarie) i utrzymuje ten stan niezależnie od zdarzeń infrastrukturalnych.

k3s to uproszczona dystrybucja Kubernetes stworzona przez Rancher Labs. Zmniejsza wymagania sprzętowe (minimalna instalacja działa na instancji z 2 GB RAM) i upraszcza instalację do pojedynczego polecenia. Jest przeznaczona dla środowisk brzegowych, IoT oraz małych klastrów. W niniejszym projekcie k3s zainstalowany na pojedynczej instancji EC2 zastępuje zarządzany klaster EKS, co pozwala ograniczyć koszty do minimum.

2.8 Infrastruktura jako kod — Terraform i Helm

Infrastruktura jako kod (ang. *Infrastructure as Code*, IaC) to praktyka definiowania i zarządzania zasobami infrastrukturalnymi za pomocą plików konfiguracyjnych przechowywanych w systemie kontroli wersji, zamiast ręcznego konfigurowania przez interfejs graficzny lub CLI [1]. Dzięki temu środowisko jest powtarzalne, audytowalne i odtwarzalne dowolnym momencie.

Terraform to narzędzie IaC firmy HashiCorp umożliwiające tworzenie, modyfikowanie i niszczenie zasobów chmurowych w deklaracyjnym języku HCL (ang. *HashiCorp Configuration Language*). Terraform utrzymuje plik stanu (`terraform.tfstate`) opisujący aktualną konfigurację infrastruktury, co pozwala wykrywać i stosować wyłącznie te zmiany, które są niezbędne. W projekcie Terraform zarządza zasobami AWS: siecią VPC, instancją EC2, zasobnikami S3 i rejestrem ECR.

Helm to menedżer pakietów dla Kubernetes, analogiczny do apt czy pip w świecie aplikacji [7]. Pakiet Helm (ang. *chart*) zawiera szablony zasobów Kubernetes parametryzowane wartościami z pliku `values.yaml`. Umożliwia to instalację, aktualizację i usuwanie złożonych aplikacji wieloskładnikowych jednym poleceniem, z pełną historią wdrożeń i możliwością cofnięcia (`rollback`). W projekcie Helm zarządza wdrożeniem API modelu oraz stosu monitorującego na klastrze k3s.

2.9 Wersjonowanie danych — DVC

DVC (ang. *Data Version Control*) to narzędzie umożliwiające wersjonowanie dużych plików binarnych (obrazów, wag modeli, zbiorów danych) w sposób spójny z Git. Zamiast przechowywać dane w repozytorium Git (co jest niepraktyczne przy dużych plikach), DVC przechowuje jedynie lekkie pliki metadanych (`.dvc`), a rzeczywiste dane trafiają do zewnętrznego magazynu — w tym projekcie do Amazon S3 [9].

Zasada działania jest analogiczna do Git: każda wersja danych ma swój hash, można cofać się do poprzednich wersji poleceniem `dvc checkout`, a polecenie `dvc pull` pobiera dane odpowiadające bieżącemu stanowi repozytorium. Dzięki temu możliwe jest odtworzenie dokładnego eksperymentu: tej samej wersji kodu, danych i konfiguracji.

2.10 Śledzenie eksperymentów — MLflow

MLflow to platforma open-source do zarządzania cyklem życia modeli ML. Składa się z czterech głównych komponentów:

- **Tracking** — rejestrowanie parametrów, metryk i artefaktów każdego uruchomienia eksperymentu,
- **Projects** — standaryzacja i reprodukowalność kodu ML,
- **Models** — zunifikowany format pakowania modeli,
- **Model Registry** — centralny rejestr wersji modeli z etapami cyklu życia (Staging, Production, Archived).

Model Registry jest kluczowym elementem w kontekście bramki jakości. Po każdym treningu nowa wersja modelu jest rejestrowana w MLflow. Bramka jakości pobiera metryki modelu aktualnie w etapie Production i porównuje z nowym wynikiem. Tylko model z lepszymi metrykami zostaje awansowany do Production [17].

2.11 Monitoring — Prometheus i Grafana

Prometheus to system monitorowania i alertowania oparty na modelu *pull* [18]. Regularnie pobiera (ang. *scrape*) metryki z endpointów HTTP udostępnianych przez monitorowane usługi i przechowuje je w wbudowanej bazie szeregów czasowych. Metryki opisywane są etykietami (ang. *labels*), co pozwala na elastyczne filtrowanie i agregację.

Grafana to platforma wizualizacji danych, która integruje się z wieloma źródłami, w tym z Prometheus. Pozwala tworzyć interaktywne dashboardy z wykresami, wskaźnikami i alertami. W projekcie Grafana prezentuje metryki działającego API modelu: liczbę predykcji, czasy odpowiedzi, zużycie zasobów i liczbę błędów. Monitoring działa jako opcjonalny stos uruchamiany na tym samym klastrze k3s co API modelu [12].

3 Architektura Systemu

Niniejszy rozdział opisuje zaprojektowaną architekturę systemu: wybrane komponenty, ich wzajemne powiązania i przesłanki kluczowych decyzji projektowych. Szczegóły implementacji poszczególnych elementów zawarte są w rozdziałach 5 i 6.

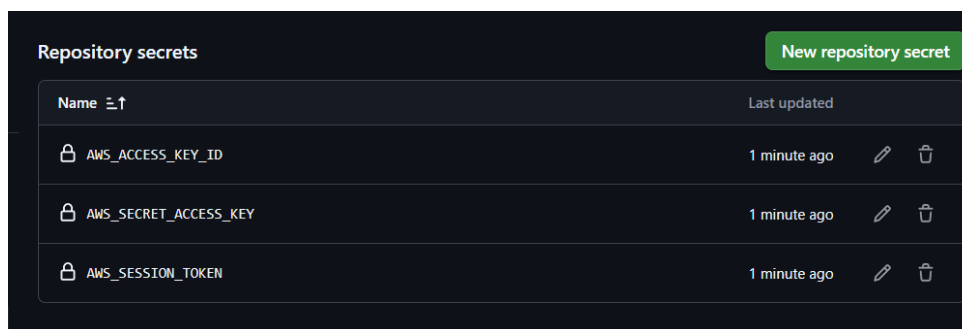
3.1 Cele i ograniczenia projektowe

Projektując system, kierowano się pięcioma celami:

1. **Pełna automatyzacja** — użytkownik jedynie będzie używać `git push` lub uruchamiać aplikacje; walidacja, trening, ocena jakości i wdrożenie odbywają się bez interwencji.
2. **Reprodukowalność** — każda wersja modelu musi być powiązana z wersją kodu, wersją danych i zapisanymi metrykami.
3. **Kontrola kosztów** — projekt realizowany w ramach konta AWS Academy z budżetem 50 USD; zasoby uruchamiane tylko gdy potrzebne.
4. **Kontrola czasu** — trening nie może przekraczać 2,5 h, aby zmieścić się w 4-godzinny oknie sesji laboratorium AWS.
5. **Porównywalność** — system musi być zbudowany w sposób umożliwiający zebranie wymierzanych danych w celu porównania z podejściem manualnym.

Ograniczenia budżetowe wykluczyły usługi o wysokim koszcie stałym: EKS (\$150/mies.), RDS (\$30/mies.), NAT Gateway (\$32/mies.) i ALB (\$18/mies.) — zastąpione przez k3s na EC2, SQLite i NodePort.

Dodatkowym ograniczeniem środowiska AWS Academy jest brak możliwości tworzenia ról IAM oraz konfiguracji federacji tożsamości OIDC. Mechanizmów, które w standardowych środowiskach produkcyjnych pozwalają GitHub Actions uwierzytelniać się w AWS bez przechowywania długoterminowych kluczy. W konsekwencji tymczasowe klucze sesji (generowane przy każdym uruchomieniu laboratorium) przechowywane są jako zaszyfrowane sekrety repozytorium GitHub i wstrzykiwane do zmiennych środowiskowych workflow podczas każdego uruchomienia [6]. Sekrety są rotowane ręcznie przy każdym odnowieniu sesji, co stanowi kompromis akceptowalny w kontekście projektu badawczego (rysunek 3).



Rys. 3 Konfiguracja sekretów GitHub Actions przechowujących tymczasowe poświadczenia AWS

3.2 Struktura repozytoriów

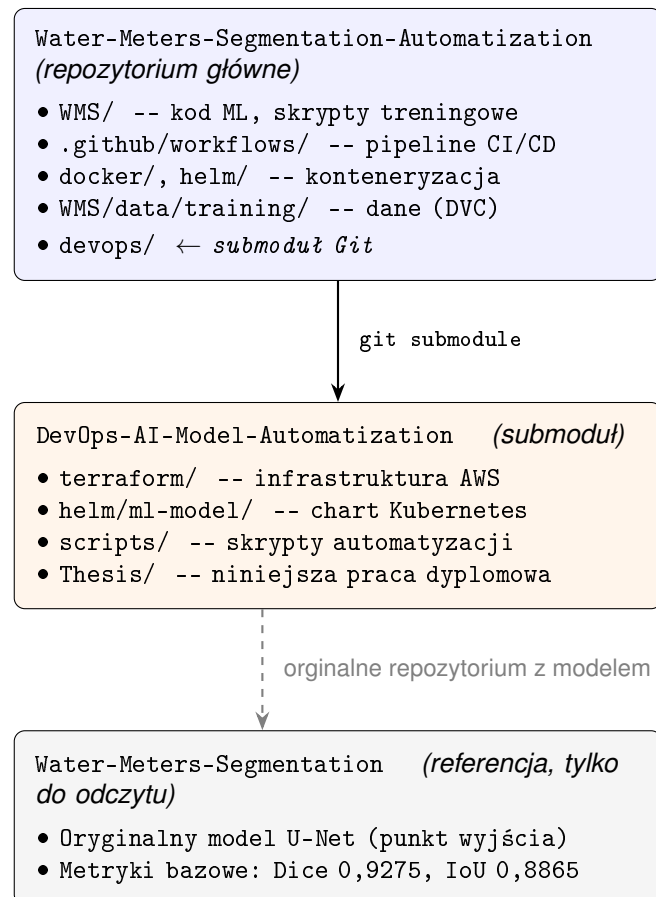
Projekt korzysta z dwóch repozytoriów Git połączonych mechanizmem submodułów:

- **Water-Meters-Segmentation-Automatization** — repozytorium główne, zawiera kod ML,

metadane danych treningowych śledzone przez DVC, workflow GitHub Actions, konfigurację Docker i Helm oraz skrypty pomocnicze. Całość procesu CI/CD wyzwalana jest w tym repozytorium.

- **DevOps-AI-Model-Automatization** — repozytorium infrastruktury, dołączone jako submoduł pod ścieżką `devops/`; zawiera moduły Terraform, skrypty konfiguracyjne EC2 oraz niniejszą pracę dyplomową.

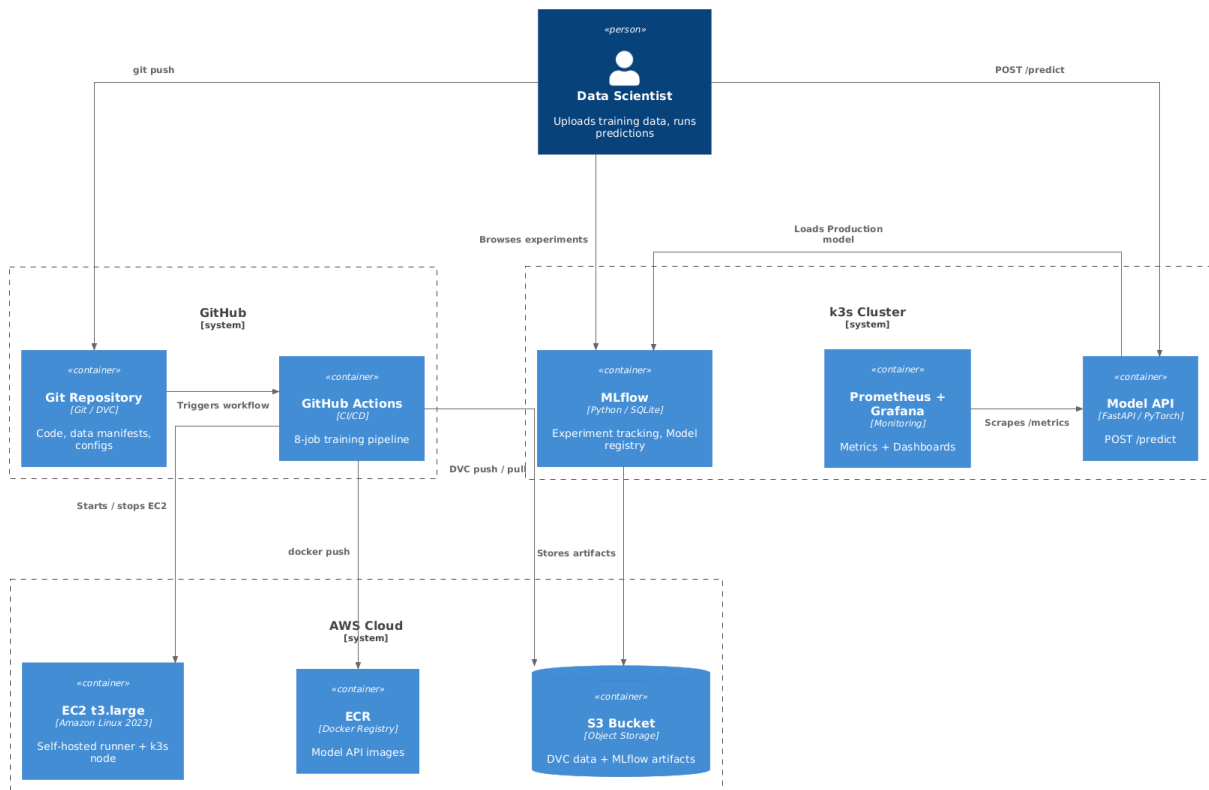
Podział na dwa repozytoria wynika z różnego rytmu zmian: kod ML i dane zmieniają się często i wyzwalają automatyczne procesy, natomiast definicja infrastruktury zmienia się rzadko i jest zarządzana ręcznie. Dzięki mechanizmowi submodułów konkretna wersja infrastruktury jest powiązana z konkretną wersją kodu aplikacji, co zachowuje pełną spójność środowiska.



Rys. 4 Struktura repozytoriów projektu: repozytorium główne zawiera infrastrukturę jako submoduł Git; repozytorium referencyjne dostarcza oryginalny model i metryki bazowe

3.3 Warstwy architektury

System zorganizowany jest w cztery logiczne warstwy odpowiadające granicom widocznym na diagramie C4 (rysunek 5): warstwa użytkownika, warstwa kontroli wersji i automatyzacji, warstwa infrastruktury chmurowej oraz warstwa platformy ML i serwowania.



Rys. 5 Architektura systemu w modelu C4. Cztery warstwy i ich komponenty.

3.3.1 Warstwa użytkownika

Punkt wejścia do systemu stanowi maszyna lokalna użytkownika (Data Scientist). Interakcja z całym pipeline'em sprowadza się do dwóch operacji: `git commit` i `git push` przy dodawaniu nowych danych treningowych oraz uruchomienia skryptu `predicts.py` przy lokalnym wykonywaniu predykcji na nowych obrazach. Zainstalowany hook `Git pre-push` [3] automatycznie przechwytuje push zawierający dane treningowe i kieruje go na oddzielną gałąź `data/TIMESTAMP`, inicjując dalszy pipeline bez konieczności jakiegokolwiek dodatkowej akcji ze strony użytkownika.

3.3.2 Warstwa kontroli wersji i automatyzacji

Centralnym punktem systemu jest platforma GitHub łącząca kontrolę wersji kodu z automatyzacją pipeline'u. Repozytorium Git [3] przechowuje kod ML, definicje workflow i konfigurację wdrożenia (Helm charts), natomiast binarne pliki danych treningowych przechowywane są w Amazon S3 i śledzone przez DVC [9] sprawia, że w repozytorium zapisywane są wyłącznie lekkie pliki metadanych (`.dvc`) zawierające hasze i rozmiary plików, co pozwala wersjonować duże zbiory danych bez obciążania historii Git.

GitHub Actions [6] orkiestruje cały cykl życia modelu: wykrywa pojawienie się gałęzi danych, pobiera historyczny zbiór z S3, scala go z nowo dostarczonymi plikami, uruchamia walidację, startuje infrastrukturę obliczeniową, trenuje model, ocenia go przez bramkę jakości: jeśli model się poprawił to wdraża go i scala gałąź danych z `main`. Workflow korzysta z reużywalnych podworkflow dla operacji na EC2, co eliminuje powielanie konfiguracji.

3.3.3 Warstwa infrastruktury chmurowej

Zasoby AWS tworzone są deklaratywnie za pomocą Terraform [1] i mogą być odtworzone jednym poleceniem `terraform apply`. Kluczowe zasoby to: instancja EC2 `t3.large` (8 GB RAM) ze statycznym Elastic IP, dwa zasobniki S3 (dane DVC i artefakty MLflow) oraz rejestr obrazów Docker Amazon ECR. Instancja EC2 pełni podwójną rolę: self-hosted runner GitHub Actions podczas treningu modelu i węzeł klastra k3s [19] podczas serwowania predykcji. Elastic IP jest niezbędne przy cyklicznym stop/start — bez niego publiczny adres IP instancji zmieniałby się po każdym starcie, uniemożliwiając stabilną konfigurację runnera i MLflow.

3.3.4 Warstwa platformy ML i serwowania

Na klastrze k3s [19] uruchomionym na instancji EC2 działają cztery komponenty. MLflow pełni rolę centralnego rejestru eksperymentów i modeli. Każde uruchomienie treningu loguje parametry, metryki i artefakty, a model w stadium Production stanowi punkt odniesienia dla bramki jakości [17]. API modelu zbudowane na FastAPI i zarządzane przez Helm [7] udostępnia endpoint `/predict` przyjmujący obrazy i zwracający maski segmentacyjne; model pobierany jest z rejestru MLflow przy starcie kontenera, co oddziela wersję obrazu Docker od wersji modelu. Konteneryzacja przez Docker [14] zapewnia identyczne środowisko uruchomieniowe niezależnie od hosta. Stos monitoringu Prometheus [18] + Grafana zbiera metryki działania API (liczba predykcji, czasy odpowiedzi, błędy) i prezentuje je na dashboardzie dostępnym przez Node-Port.

3.4 Kluczowe decyzje projektowe

Tabela 1 zestawia najważniejsze decyzje projektowe podjęte podczas budowy systemu, wraz z uzasadnieniem wyboru i odrzuconą alternatywą. Każda z nich wynika z jednego lub więcej celów wymienionych w sekcji 3.

Tabela 1 Kluczowe decyzje architektoniczne

Decyzja	Wybór	Uzasadnienie
Orchestracja kontenerów	k3s na EC2	EKS jest 15× droższy; k3s wystarcza dla jednego węzła
Backend MLflow	SQLite	RDS \$30/mies.; SQLite jest wystarczający dla małego projektu; dane Meta przechowywane na EC2
Dostęp do API	NodePort 30080	Load Balancer \$18/mies.; publiczny adres IP EC2 z Elastic IP jest wystarczający
Typ instancji EC2	t3.large (8 GB RAM)	t3.small/medium: crash OOM przy pobieraniu obrazu PyTorch 8,69 GB; t3.large jest minimum
Stop/start EC2	Automatyczny po każdym pipeline	EC2 działa tylko 3–4 h na uruchomienie; oszczędność 80% kosztów vs. ciągłe działanie

(ciąg dalszy na następnej stronie)

Decyzja	Wybór	Uzasadnienie
Dane treningowe w Git	Tylko metadane DVC	Duże pliki binarne spowalniają Git i przekraczają limity; S3 jest tańszy i skalowalny
Trening przed PR	Trening → PR (jeśli lepszy)	PR tworzony przez bota nie wyzwala innych workflow (ograniczenie GitHub); kolejność eliminuje problem
Baseline bramki jakości	Dynamiczny MLflow	z Hardkodowany baseline staje się nieaktualny po każdym treningu; MLflow zawsze zna aktualny Production
IAM dla GitHub Actions	Tymczasowe klucze sesji	AWS Academy zabrania tworzenia ról IAM; OIDC niedostępne; klucze rotowane ręcznie
Scalanie danych treningowych	GitHub Actions	Eliminuje konieczność lokalnych poświadczeń AWS u każdego dewelopera; hook pre-push tworzy jedynie gałąź danych (<10 s) bez dostępu do S3
Framework serwowania API	FastAPI + Uvicorn	Asynchroniczne I/O, automatyczna dokumentacja OpenAPI; Flask synchroniczny; TorchServe nadmiarowy dla jednego modelu
Metryki działania API	Prometheus client	Prometheus i Grafana wbudowane w stos k3s; CloudWatch generuje dodatkowe koszty; bezpłatna integracja
Obraz bazowy Docker	python:3.12-slim	Mniejszy rozmiar niż python:3.12; zależności systemowe (libgl1, libglib2.0) dodane explicite, co czyni je widocznymi i reprodukowalnymi
Artefakty MLflow	Amazon S3	Lokalny dysk EC2 jest ulotny po zatrzymaniu instancji; S3 zapewnia trwałość modeli i logów niezależnie od cyklu stop/start
System operacyjny EC2	Amazon 2023	Linux OpenSSL 3.0 kompatybilny z urllib3 v2.0 (AL2 wymagał przypinania); wsparcie do 2028; natywny dnf

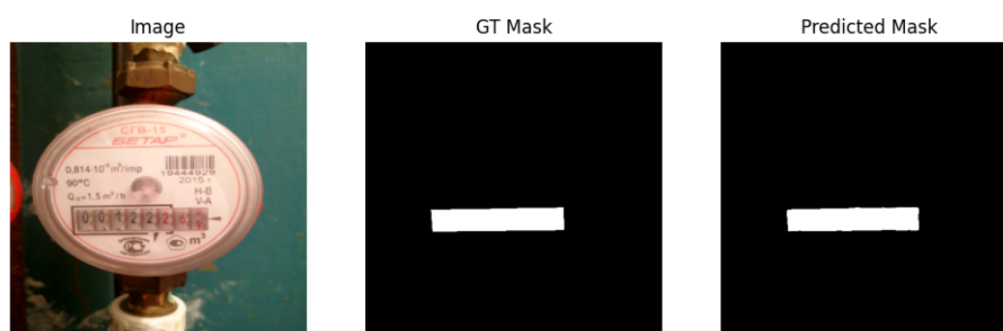
4 Model Sztucznej Inteligencji

4.1 Charakterystyka problemu segmentacji semantycznej

Model sztucznej inteligencji stanowiący podstawę niniejszego projektu został zaprojektowany do rozwiązywania zadania **segmentacji semantycznej obrazów wodomierzy**. Celem segmentacji jest przypisanie każdemu pikselowi obrazu etykiety określającej jego przynależność do jednej z wyróżnionych klas, w szczególności obszaru licznika, tarczy oraz tła.

Problem ten ma istotne znaczenie praktyczne w kontekście automatyzacji odczytu wskazań wodomierzy, gdzie poprawne wydzielenie obszaru licznika stanowi warunek konieczny dla dalszych etapów przetwarzania obrazu, takich jak rozpoznawanie cyfr lub analiza wskazań mechanicznych. W przeciwieństwie do klasycznej klasyfikacji obrazów, segmentacja semantyczna wymaga zachowania informacji przestrzennej oraz precyzyjnego odwzorowania granic obiektów, co znacząco zwiększa złożoność problemu.

Dodatkowym wyzwaniem są zmienne warunki akwizycji danych, obejmujące różnice w oświetleniu, kącie wykonania zdjęcia, jakości obrazu oraz stopniu zabrudzenia lub zużycia urządzeń pomiarowych. Czynniki te powodują, że klasyczne algorytmy przetwarzania obrazu okazują się niewystarczające, co uzasadnia zastosowanie metod opartych na uczeniu głębokim.



Rys. 6 Przykład predykcji modelu segmentacji semantycznej

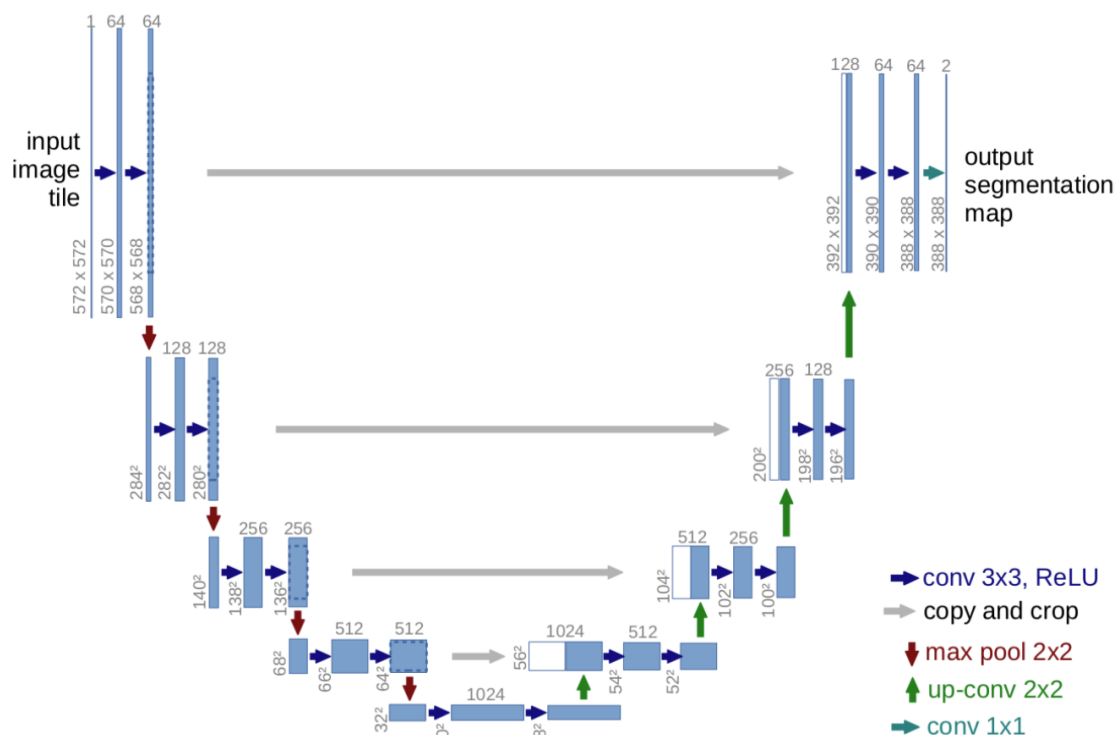
4.2 Architektura modelu

W projekcie wykorzystano konwolucyjną sieć neuronową opartą na architekturze **U-Net**, powszechnie stosowaną w zadaniach segmentacji semantycznej obrazów. Architektura ta charakteryzuje się symetryczną budową typu encoder-decoder, umożliwiającą jednocześnie wydobywanie cech semantycznych oraz zachowanie informacji przestrzennej.

Część kodująca (encoder) odpowiada za stopniowe przekształcanie obrazu wejściowego do reprezentacji o coraz wyższym poziomie abstrakcji poprzez sekwencję operacji konwolucji oraz redukcji rozdzielczości. Część dekodująca (decoder) realizuje proces rekonstrukcji obrazu wyjściowego, przywracając jego pierwotną rozdzielczość i generując maskę segmentacyjną.

Kluczowym elementem architektury są tzw. *skip connections*, umożliwiające bezpośrednie przekazywanie map cech pomiędzy odpowiadającymi sobie poziomami enkodera i dekodera. Mechanizm ten pozwala ograniczyć utratę informacji lokalnej oraz poprawić jakość segmentacji, szczególnie w obszarach granicznych pomiędzy klasami.

Zastosowana implementacja architektury U-Net została dostosowana do specyfiki problemu, w tym rozdzielczości obrazów wejściowych oraz liczby klas segmentacyjnych. Model generuje maskę wyjściową o tej samej rozdzielczości co obraz wejściowy, co ułatwia dalsze etapy przetwarzania oraz ocenę jakości predykcji.



Rys. 7 Schemat architektury U-Net [4]

4.3 Dane treningowe i przygotowanie zbioru danych

Model został wytrenowany na zbiorze danych składającym się z obrazów wodomierzy oraz odpowiadających im masek. Każda próbka zawiera obraz wejściowy w postaci fotografii oraz maskę określającą przynależność poszczególnych pikseli do zdefiniowanych klas.

Proces przygotowania danych obejmował normalizację wartości pikseli, standaryzację rozdzielczości obrazów oraz podział zbioru danych na część treningową, walidacyjną i testową. W celu zwiększenia odporności modelu na zmienność danych wejściowych zastosowano techniki augmentacji danych, takie jak losowe obroty, skalowanie, przesunięcia oraz modyfikacje jasności i kontrastu.

Zastosowanie augmentacji pozwala ograniczyć ryzyko przeuczenia modelu oraz poprawić jego zdolność do generalizacji na dane niewidziane w procesie uczenia. Istotnym aspektem projektu jest fakt, że zbiór danych oraz jego kolejne wersje podlegają wersjonowaniu, co umożliwia jednoznaczne powiązanie konkretnej wersji modelu z użytymi danymi treningowymi.

Zbiór danych wykorzystany w projekcie pochodzi z platformy Kaggle [10].

4.4 Proces uczenia i metryki jakości

Proces uczenia modelu realizowany jest w sposób nadzorowany, z wykorzystaniem funkcji straty dostosowanej do problemu segmentacji semantycznej. Jako główne metryki oceny ja-

kości modelu zastosowano **współczynnik Dice** oraz **Intersection over Union (IoU)**, które są standardowymi miarami stosowanymi w zadaniach segmentacyjnych.

Współczynnik Dice umożliwia ocenę stopnia pokrycia obszaru przewidywanego przez model z obszarem referencyjnym, natomiast IoU mierzy stosunek części wspólnej obu obszarów do ich sumy. Zastosowanie obu metryk pozwala na bardziej kompleksową ocenę jakości predykcji, szczególnie w przypadku niebalansowanych klas.

Proces uczenia realizowany jest iteracyjnie, z wykorzystaniem zbioru walidacyjnego do monitorowania jakości modelu oraz ograniczania zjawiska przeuczenia. Najlepsza wersja modelu, zgodnie z przyjętymi kryteriami jakości, zapisywana jest jako artefakt i przekazywana do dalszych etapów zautomatyzowanego pipeline'u CI/CD.

4.5 Model jako artefakt w procesie CI/CD

W kontekście niniejszej pracy model sztucznej inteligencji traktowany jest jako pełnoprawny artefakt inżynierski, podlegający wersjonowaniu, testowaniu oraz kontrolowanemu wdrażaniu. Każda wersja modelu jest jednoznacznie powiązana z wersją kodu źródłowego, zestawem danych treningowych, konfiguracją hiperparametrów oraz uzyskanymi metrykami jakości.

Takie podejście umożliwia pełną reprodukowalność wyników oraz stanowi podstawę do dalszej analizy porównawczej pomiędzy podejściem manualnym a zautomatyzowanym procesem CI/CD. Szczegółowa analiza wpływu automatyzacji na jakość, stabilność oraz koszty utrzymania modeli sztucznej inteligencji zostanie przedstawiona w kolejnych rozdziałach pracy.

water-meter-segmentation

Provide Feedback

Add Description

Share

Runs

Models

Experimental

Evaluation

Traces

metrics.rmse >= 0.8

Datasets

Sort: Creation time

Columns

Model attributes		Model attributes		No dataset		Parameters					
Model name	Status	Created	Logged from	Source run	Registered models	final_test_dice	final_test_iou	final_test_hausdorff	batch...	color_jitter_prob	early_stopping
model	Ready	1 day ago	train.py	9c20b18-unknown	water-meter-segmentation v20	0.8134316205978394	0.7409412860870361	59.99671981172094	4	0.3	5
model	Ready	4 days ago	train.py	04a02a8-unknown	water-meter-segmentation v18	0.599755048751831	0.5507521629333496	226.5247494011263	4	0.3	5
model	Ready	5 days ago	train.py	1d8da44-unknown	water-meter-segmentation v16	0.4824694991117554	0.4517785310745239	94.33213923316785	4	0.3	5
model	Ready	5 days ago	train.py	287f548-unknown	water-meter-segmentation v14	0.4710100591182709	0.4060139060020447	181.28396784630462	4	0.3	5
model	Ready	5 days ago	train.py	287f548-unknown	water-meter-segmentation v13	0.07412752509117126	0.03881072998046875	148.55985911849012	4	0.3	5
model	Ready	5 days ago	train.py	06f9c9-unknown	water-meter-segmentation v12	0.0042421105317771435	0.0021263936068862677	358.614131846163	4	0.3	5
model	Ready	5 days ago	train.py	3a5ede1-unknown	water-meter-segmentation v11	0.04485442116856575	0.023298226296901703	355.3104910629958	4	0.3	5
model	Ready	5 days ago	train.py	3a5ede1-unknown	water-meter-segmentation v10	0.04485442116856575	0.023298226296901703	355.3104910629958	4	0.3	5
model	Ready	5 days ago	train.py	bd6c167-unknown	water-meter-segmentation v9	0.31157487630844116	0.24609875679016113	329.85860615040275	4	0.3	5
model	Ready	5 days ago	train.py	9289672-unknown	water-meter-segmentation v8	0.053569596260786057	0.027751922607421875	336.6613202864257	4	0.3	5
model	Ready	5 days ago	train.py	0022a5c-unknown	water-meter-segmentation v7	0.053569596260786057	0.027751922607421875	336.6613202864257	4	0.3	5
model	Ready	5 days ago	train.py	67b15f5-unknown	water-meter-segmentation v5	0.3125496804714203	0.18522007763385773	272.5949375905576	4	0.3	5
model	Ready	5 days ago	train.py	e5f7b1c-unknown	water-meter-segmentation v3	3.5688793587063117e-10	3.5688793587063117e-10	724.0773439350247	4	0.3	5
model	Ready	5 days ago	train.py	ced904a-unknown	water-meter-segmentation v1	3.538570270134045e-10	3.538570270134045e-10	409.21632421006865	4	0.3	5

Rys. 8 Wersjonowanie modeli w rejestrze MLflow

5 Pipeline CI/CD

Rozdział opisuje implementację zautomatyzowanego pipeline CI/CD dla modeli uczenia maszynowego. Prezentowane rozwiązanie realizuje przepływ: dodanie danych treningowych przez użytkownika wyzwala sekwencję kroków; walidację, trening, ocenę jakości i wdrożenie, bez konieczności ręcznej interwencji.

5.1 Hook pre-push

Pierwszym elementem pipeline jest hook Git pre-push [3], instalowany lokalnie na maszynie użytkownika przez skrypt `devops/scripts/install-git-hooks.sh`. Hook uruchamia się automatycznie przed każdym `git push` i sprawdza, czy w zestawie zmian znajdują się pliki w katalogach `WMS/data/training/images/` lub `WMS/data/training/masks/`.

Jeśli zmiany dotyczą danych treningowych, hook:

1. tworzy nową gałąź o nazwie `data/TIMESTAMP` (np. `data/20260215-143022`),
2. commituje do niej wyłącznie nowe pliki z katalogów danych,
3. przekierowuje push na tę gałąź zamiast na `main`.

Całość wykonuje się w mniej niż 10 sekund, ponieważ hook nie wymaga połączenia z AWS ani lokalnych poświadczeń chmurowych. Merge danych z historycznym zbiorem w S3 odbywa się po stronie GitHub Actions.

```
Training data changes detected. Creating data branch...
🔨 Creating branch: data/20260217-013810
🚀 Pushing to data/20260217-013810...
🔍 Checking for training data changes...
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 584 bytes | 584.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'data/20260217-013810' on GitHub by visiting:
remote:   https://github.com/Rafallost/Water-Meters-Segmentation-Autimatization/pull/new/data/20260217-013810
remote:
To https://github.com/Rafallost/Water-Meters-Segmentation-Autimatization.git
* [new branch]    data/20260217-013810 -> data/20260217-013810
branch 'data/20260217-013810' set up to track 'origin/data/20260217-013810'.

✅ Success! Your changes are now on branch: data/20260217-013810

Next steps:
1. GitHub Actions will validate your data
2. If valid, a Pull Request will be created automatically
3. Training will run automatically
4. If model improves, PR will be auto-approved
5. You (or admin^*) can then merge the PR

error: failed to push some refs to 'https://github.com/Rafallost/Water-Meters-Segmentation-Autimatization.git'
PS D:\school\bsc\Repositories\Water-Meters-Segmentation-Autimatization> |
```

Rys. 9 Wynik wypchania nowych danych na gałąź `data/TIMESTAMP`

Po zakończeniu działania hooka Git wyświetla komunikat `error: failed to push some refs`. Jest to zachowanie oczekiwane, nie błąd. Hook celowo kończy się kodem niezerowym, aby zablokować oryginalny push do `main`. Dane zostały już pomyślnie wypchnięte na gałąź `data/TIMESTAMP` i GitHub Actions może przejąć kontrolę.

5.2 Struktura workflow GitHub Actions

Główny plik automatyzacji projektu to `.github/workflows/training-data-pipeline.yaml` [6]. Definiuje on trigger, konfigurację współbieżności, uprawnienia i osiem jobów składających się na pipeline.

Trigger

Workflow uruchamia się wyłącznie na push do gałęzi pasujących do wzorca `data/**`. Oznacza to, że kod przesyłany do `main` lub innych gałęzi nie inicjuje treningu — pipeline startuje tylko gdy hook `pre-push` przekieruje dane na gałąź `data/TIMESTAMP`:

```
on:
  push:
    branches:
      - 'data/**'

concurrency:
  group: training-pipeline
  cancel-in-progress: false

permissions:
  contents: write
  pull-requests: write
```

Runnery

Większość jobów uruchamia się na środowisku zarządzanym przez GitHub (`runs-on: ubuntu-latest`), co nie wymaga utrzymania własnej infrastruktury i nie generuje kosztów podczas oczekiwania. Wyjątkiem są joba `train` i `deploy` — wykonują się na `runs-on: self-hosted`, czyli bezpośrednio na instancji EC2. Taki podział eliminuje transfer danych treningowych przez sieć publiczną: dane pozostają na EC2 przez cały czas treningu.

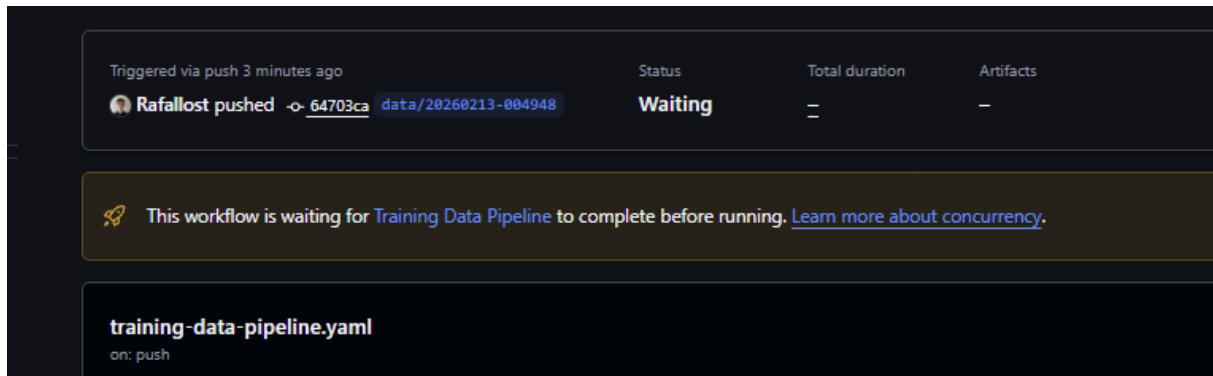
Komunikacja między jobami

GitHub Actions umożliwia przekazywanie wartości między jobami przez mechanizm *outputs*. Job `merge-and-validate` eksponuje wyniki walidacji (`validation_passed`, liczba obrazów), a job `train` eksponuje flagę `improved`, na podstawie której warunkowane są dalsze joby. Artefakty GitHub Actions służą natomiast do przekazywania samych plików danych (scalony zbiór) między jobem `merge-and-validate` a jobem `train`, który wykonuje się na innym runnerze.

Współbieżność

Klucz `concurrency.cancel-in-progress: false` sprawia, że jeśli użytkownik wypchnął dane

kilkakrotnie zanim poprzednie uruchomienie zdążyło się zakończyć, kolejne uruchomienia nie są anulowane tylko trafiają do kolejki i wykonują się po zakończeniu poprzednich. Podejście to chroni przed sytuacją, w której późniejszy push nadpisze wyniki wcześniejszego treningu, zanim zostanie on oceniony przez bramkę jakości.



Rys. 10 Kolejowanie uruchomień workflow w GitHub Actions. Nowe uruchomienie czeka na zakończenie poprzedniego.

5.3 Walidacja danych

Po wypchnięciu gałęzi danych uruchamia się pierwszy job workflow `training-data-pipeline.yaml:merge-and-validate` [6]. Składa się on z dwóch faz.

Faza scalania

Job pobiera istniejące dane z Amazon S3 (za pomocą poświadczeń przechowywanych w sekretach GitHub), scala je z nowo dostarczonymi plikami i aktualizuje manifesty DVC. Scalanie jest idempotentne — pliki o identycznej nazwie i sumie kontrolnej nie są duplikowane.

Faza walidacji

Skrypt `data-qa.py` weryfikuje scalony zbiór danych pod kątem czterech kryteriów:

- **kompletność par** — dla każdego obrazu w katalogu `images/` musi istnieć maska o tej samej nazwie w `masks/` (porównanie po *stem* nazwy pliku),
- **rozdzielczość** — każdy plik obrazu i maski musi mieć rozdzielczość 512×512 px,
- **binarność masek** — maski mogą zawierać wyłącznie wartości 0 i 255,
- **format pliku** — akceptowane są formaty `.jpg` oraz `.png`.

Wynik walidacji jest publikowany jako komentarz w pull requestcie. W przypadku błędów komentarz zawiera listę plików niezgodnych z wymaganiami, a dalsze joby są pomijane (warunkowanie przez `needs` i `if: needs.merge-and-validate.outputs.valid == 'true'`).

```
python devops/scripts/data-qa.py WMS/data/training/ --output report.json
```

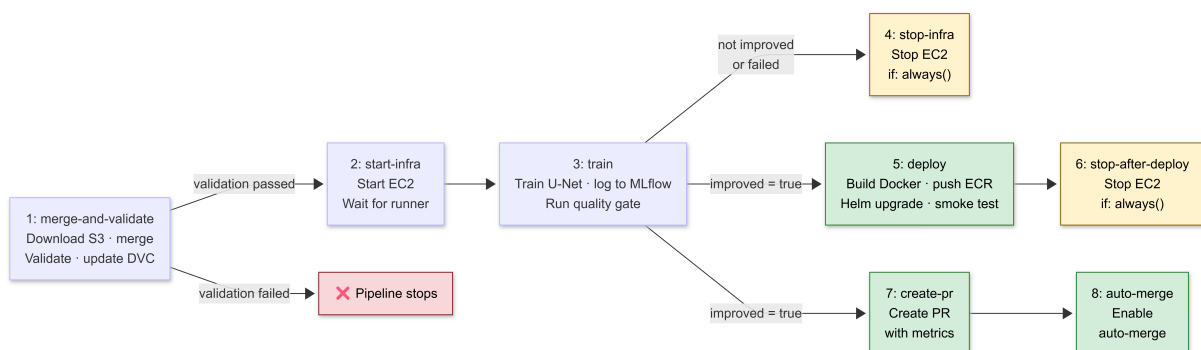
```
Run Data QA Validation

1 ▶ Run echo "Running data quality assurance..."
26 Running data quality assurance...
27 =====
28 ✖ DATA QA FAILED
29 =====
30 Images: 11 | Masks: 11 | Valid pairs: 11
31
32 Errors: 3
33 - id_1_value_13_116: Resolution mismatch
34 - id_2_value_495_341: Resolution mismatch
35 - id_3_value_366_885: Resolution mismatch
36 Error: Process completed with exit code 1.
```

Rys. 11 Przykład komentarza z wynikiem walidacji danych. W tym przypadku wykryto złą rozdzielczość, co skutkuje zatrzymaniem dalszych kroków pipeline.

5.4 Pipeline treningowy

Główny workflow `training-data-pipeline.yaml` składa się z ośmiu jobów. Joby 1–3 wykonywane są sekwencyjnie. Po zakończeniu treningu workflow rozgałęzia się w zależności od wyniku bramki jakości: jeśli model nie poprawił metryk, uruchamia się `stop-infra` (job 4) i pipeline kończy działanie. Jeśli model się poprawił, joby `deploy` (job 5) i `create-pr` (job 7) uruchamiają się równoległe i wzajemnie się nie blokują. Priorytety są celowe: dodanie ulepszanego modelu i scalenie z `main` jest ważniejsze niż wdrożenie, dlatego ewentualna awaria wdrożenia nie blokuje propagacji zatwierdzonego zbioru do repozytorium. Wdrożenie można w każdej chwili ponowić przez workflow `release-deploy.yaml` lub `deploy-model.yaml`. Job 6 zatrzymuje EC2 po wdrożeniu, a job 8 finalizuje scalenie pull requesta.



Rys. 12 Sekwencja jobów w workflow `training-data-pipeline.yaml`

Job 1: merge-and-validate

Omówiony we wcześniejszym fragmencie. Pobiera istniejące dane z S3, scala je z nowo dostarczonymi plikami, uruchamia skrypt `data-qa.py`, aktualizuje manifesty DVC i przesyła scalony zbiór jako artefakt GitHub Actions.

Job 2: start-infra

Wykonuje się na hoście GitHub (nie na EC2) i jest odpowiedzialny wyłącznie za uruchomienie infrastruktury. Wywołuje AWS CLI (`aws ec2 start-instances`) z identyfikatorem instancji zapisanym w sekretach repozytorium. Po wysłaniu polecenia co 30 sekund sprawdza API GitHub, czy self-hosted runner przypisany do tej instancji pojawił się w stanie *online* i czeka maksymalnie 10 minut. Dzięki temu każdy kolejny job pewnie trafi na już działającą instancję.

Job 3: train

Jedyny job wykonywany na self-hosted runnerze, czyli bezpośrednio na instancji EC2. Pobiera scalony zbiór danych z artefaktu GitHub Actions, uruchamia skrypt `train.py`, który inicjalizuje model U-Net z seedem równym numerowi uruchomienia (`run_number`), trenuje przez zdefiniowaną liczbę epok i loguje metryki `val_dice` oraz `val_iou` do MLflow. Po zakończeniu treningu skrypt `quality-gate.py` pobiera metryki aktualnego modelu Production z rejestru MLflow, porównuje je z wynikami nowego treningu i ustawia output `improved=true` lub `improved=false`, sterując dalszym przebiegiem workflow.

Job 4: stop-infra

Zatrzymuje instancję EC2 gdy trening zakończył się błędem lub model nie poprawił wyników (`if: always() && (train.result != 'success' || improved != 'true')`). Jest to reużywalny workflow wywołujący `aws ec2 stop-instances`. Dzięki warunkowi `always()` job uruchamia się nawet gdy poprzednie joby zakończyły się wyjątkiem, eliminując ryzyko pozostawienia uruchomionej instancji i nieoczekiwanych kosztów.

Job 5: deploy

Uruchamia się wyłącznie jeśli model spełnił kryteria bramki jakości (`if: improved == 'true'`). Wykonuje się na self-hosted runnerze na EC2. Buduje obraz Docker na podstawie pliku `docker/Dockerfile.serve`, taguje go numerem wersji i hashem commita, uwierzytelnia się w Amazon ECR i wgrywa obraz. Następnie wywołuje `helm upgrade`, który zaktualizuje deployment na klastrze k3s do nowej wersji obrazu. Kontener przy starcie pobiera model ze stadium Production z rejestru MLflow, co oddziela wersję obrazu od wersji modelu. Po wdrożeniu wykonywany jest smoke test sprawdzający dostępność endpointów `/health`, `/predict` i `/metrics`.

Job 6: stop-after-deploy

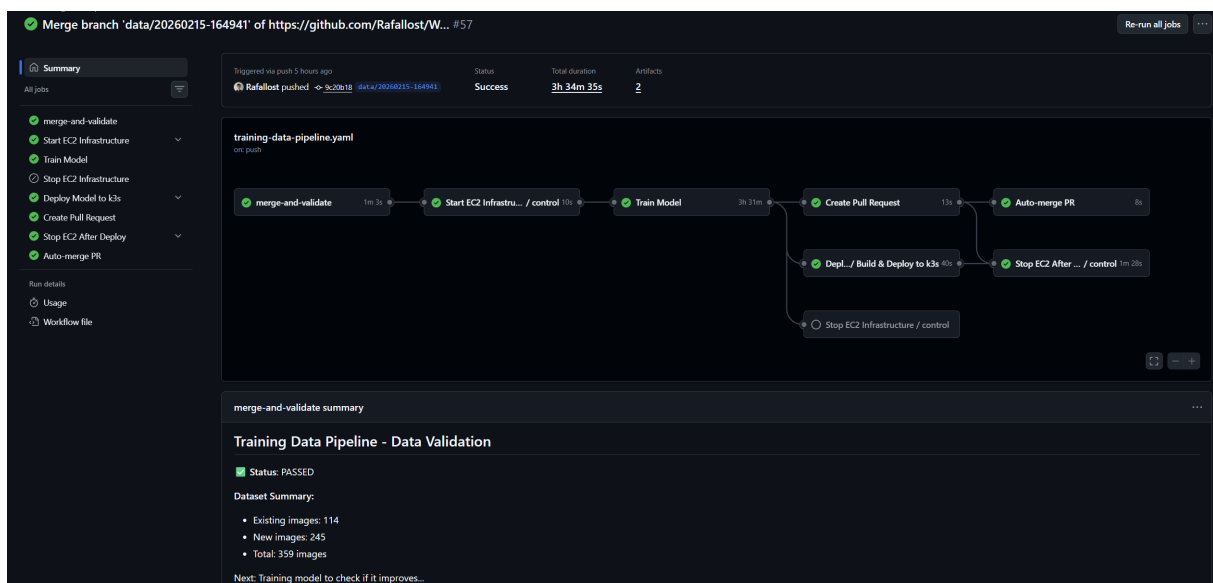
Zatrzymuje instancję EC2 po zakończeniu joba `deploy` (`if: always()`). Stanowi odpowiednik `stop-infra` dla ścieżki, gdy model się poprawił. Istnieje jako osobny job, ponieważ `deploy` korzysta z runnera na EC2. Instancja musi pozostać uruchomiona do końca wdrożenia i może zostać zatrzymana dopiero po jego zakończeniu.

Job 7: create-pr

Uruchamia się równolegle z jobem `deploy`, jeśli model spełnił kryteria bramki jakości. Tworzy pull request z gałęzi danych do `main` z automatycznie generowanym opisem zawierającym zestawienie metryk: wartości `val_dice` i `val_iou` nowego modelu oraz poprzedniego `baseline`, a także link do uruchomienia MLflow. Pull request scala zaktualizowane manifesty DVC (`images.dvc`, `masks.dvc`) z gałęzią główną, trwale łącząc wersję kodu z wersją danych.

Job 8: auto-merge

Włącza automatyczne scalanie pull requesta przez API GitHub (`gh pr merge -auto`). Scalenie następuje natychmiast po pozytywnym przejściu statusowych sprawdzeń wymaganych przez reguły ochrony gałęzi. W efekcie, jeśli model się poprawił to cały cykl od wypchnięcia danych do scalenia z `main` odbywa się bez żadnej ręcznej interwencji.



Rys. 13 Udany przebieg pipeline CI/CD z automatycznym wdrożeniem nowej wersji modelu

5.5 Bramka jakości

Bramka jakości jest mechanizmem decydującym, czy nowy model jest wystarczająco dobry, żeby trafić do produkcji. Implementacja korzysta z API MLflow Model Registry [17] (`quality-gate.py`). Skrypt łączy się z MLflow przez `MlflowClient` i pobiera metryki aktualnego modelu Production:

Pobieranie baseline

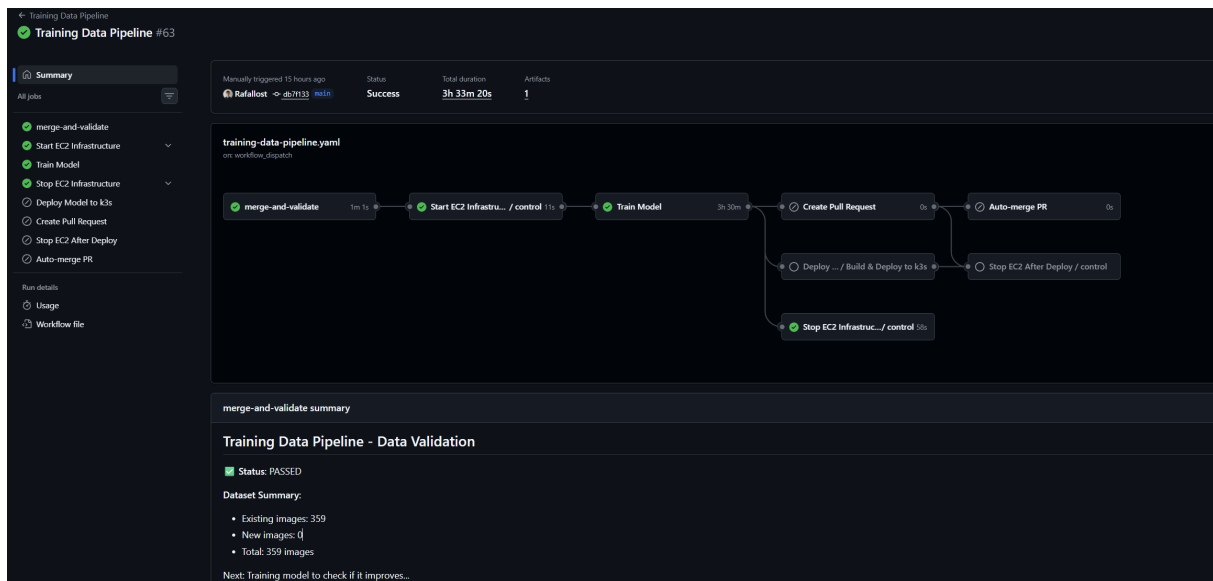
```
client = MlflowClient(tracking_uri=mlflow_uri)
versions = client.get_latest_versions(
    "water-meter-segmentation",
    stages=["Production"]
)
```


Logika decyzyjna

Model jest uznany za ulepszony, jeśli obie metryki walidacyjne są wyższe od baseline:

- `val_dice > baseline_dice`
- `val_iou > baseline_iou`

Jeśli warunek jest spełniony, skrypt promuje nowy model do etapu Production w MLflow i archiwizuje poprzednią wersję. Wynik jest przekazywany jako output do GitHub Actions (`echo "improved=true" >> $GITHUB_OUTPUT`).



Rys. 14 Przykład z nieudanego przebiegu pipeline CI/CD — model nie spełnia kryteriów bramki jakości

```
358
359 TRAINING (seed=63)
360
361
362 Results: Dice=0.8067, IoU=0.7286
363 Baseline: Dice=0.8134, IoU=0.7409
364 Improved: ❌ NO
365
366
367 QUALITY GATE RESULTS
368
369 {
370   "run_id": "8a5c55970150428294215e20d7a0db9f",
371   "seed": 63,
372   "metrics": {
373     "dice": 0.8066869974136353,
374     "iou": 0.7286145687103271
375   },
376   "baseline": {
377     "dice": 0.8134316205978394,
378     "iou": 0.7409412860870361
379   },
380   "improved": false
381 }
```

Rys. 15 Wynik bramki jakości — model nie spełnił kryteriów, więc nie został wdrożony

Train Model summary

Training Results

❌ Model Did Not Improve

Metric	Result	Production Baseline
Dice	0.8066869974136353	0.8134
IoU	0.7286145687103271	0.7409

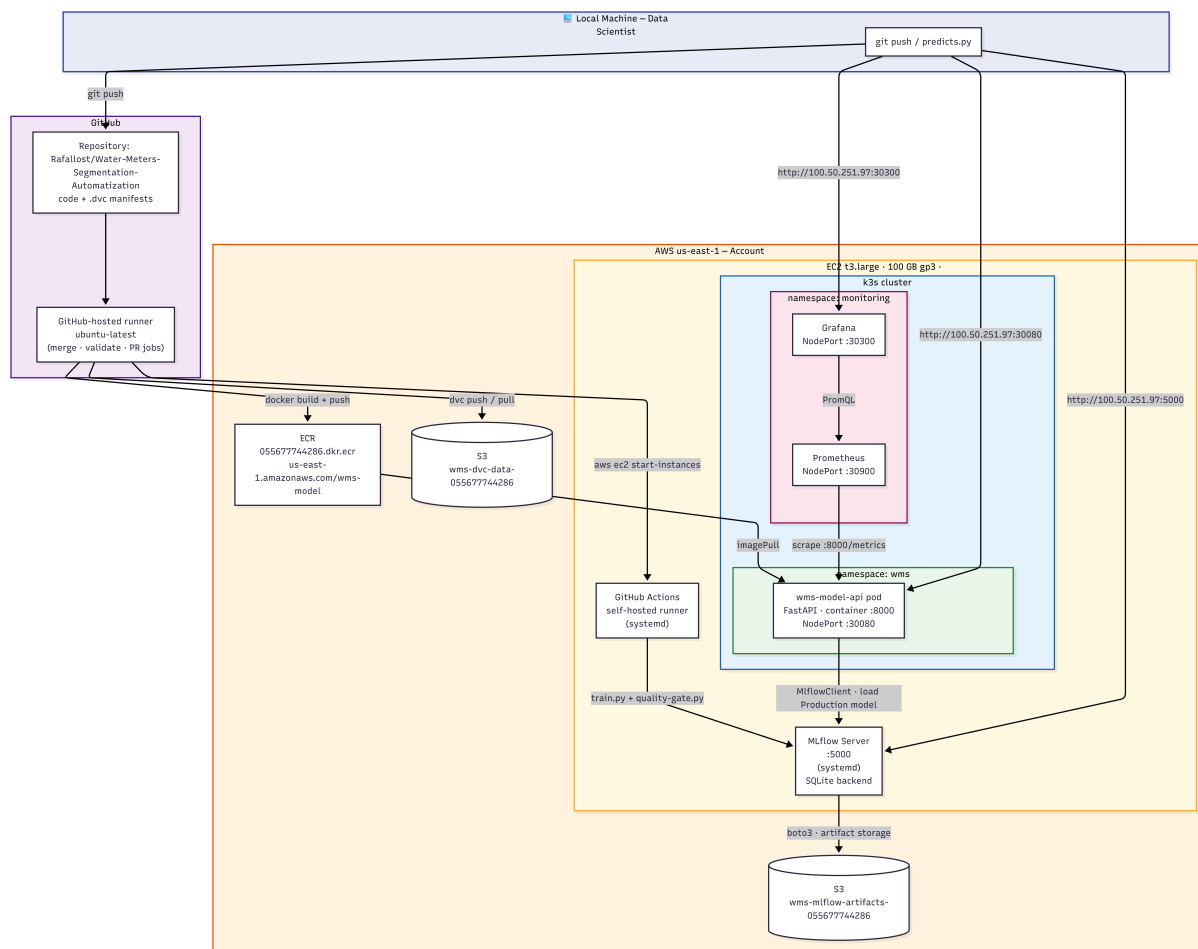
No PR will be created.

[Job summary generated at run-time](#)

Rys. 16 Podsumowanie metryk modelu, który nie spełnił kryteriów bramki jakości

6 Wdrożenie Systemu

Rozdział opisuje infrastrukturę, na której działa system, sposób jej konfiguracji oraz scenariusze użycia z perspektywy użytkownika.



Rys. 17 Diagram wdrożenia systemu — konkretne zasoby AWS, adresy i porty

6.1 Infrastruktura Terraform

Infrastruktura AWS jest zdefiniowana jako kod przy użyciu Terraform [1]. Definicje znajdują się w katalogu `devops/terraform/` i obejmują wszystkie zasoby potrzebne do działania projektu. Zastosowane podejście IaC (Infrastructure as Code) pozwala odtworzyć całe środowisko jedną komendą `terraform apply`, bez ręcznego klikania w konsoli AWS.

Pliki konfiguracyjne są podzielone według odpowiedzialności:

- `main.tf` — zasoby EC2, VPC, sieci,
- `s3.tf` — zasobniki S3 i uprawnienia,
- `ecr.tf` — rejestr obrazów Docker,
- `iam.tf` — role i polityki IAM,
- `variables.tf` — deklaracje zmiennych z wartościami domyślnymi,
- `terraform.tfvars` — konkretne wartości zmiennych (region, typ instancji, IP operatora).

Tworzone zasoby:

- **VPC i podsieć publiczna** — prosta sieć z bezpośrednim dostępem do internetu przez

- Internet Gateway; eliminuje potrzebę NAT Gateway (oszczędność ~\$32/miesiąc),
- **Security Group** — reguły ruchu przychodzącego i wychodzącego opisane w sekcji 6.3,
- **Instancja EC2** — typ `t3.large` (8 GB RAM, 2 vCPU); konfigurowalny przez zmienną `instance_type` w `terraform.tfvars`,
- **Amazon S3** — dwa zasobniki: `wms-dvc-data-*` dla danych treningowych DVC i `wms-mlflow-artifacts-*` dla artefaktów MLflow,
- **Amazon ECR** — rejestr obrazów Docker dla API modelu,
- **IAM Role** — rola instancji EC2 z uprawnieniami do odczytu i zapisu S3 oraz ECR; rola jest przypisywana do instancji, co eliminuje potrzebę przechowywania kluczy AWS na maszynie.

Infrastruktura uruchamiana jest jednorazowo poleceniem `terraform apply` (rys. 18), nie tworzona od nowa przy każdym treningu. Zasoby statyczne (VPC, S3, ECR, IAM) istnieją przez cały czas życia projektu. Instancja EC2 jest natomiast zatrzymywana po każdym uruchomieniu pipeline'u i wznawiana tylko na czas treningu lub wdrożenia, co istotnie obniża koszty eksploatacji.

```
module.vpc.aws_route_table_association.public: Creation complete after 1s [id=rtbassoc-0ee193c847ed36b1f]
module.ec2.aws_instance.k3s: Still creating... [00m10s elapsed]
module.ec2.aws_instance.k3s: Creation complete after 15s [id=i-0abe1fe1833dbecde]
module.ec2.aws_instance.k3s: Creation complete after 15s [id=i-0abe1fe1833dbecde]
module.ec2.aws_eip.k3s: Creating...
module.ec2.aws_eip.k3s: Creation complete after 4s [id=eipalloc-07c6c1e01190ef221]
module.ec2.aws_eip.k3s: Creation complete after 4s [id=eipalloc-07c6c1e01190ef221]

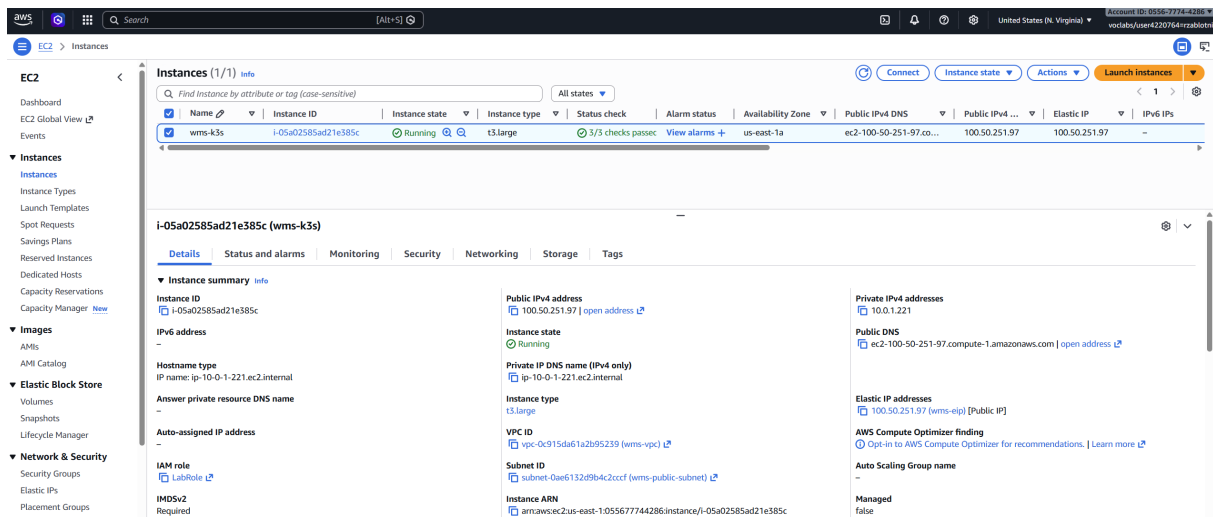
Apply complete! Resources: 16 added, 0 changed, 0 destroyed.

Outputs:

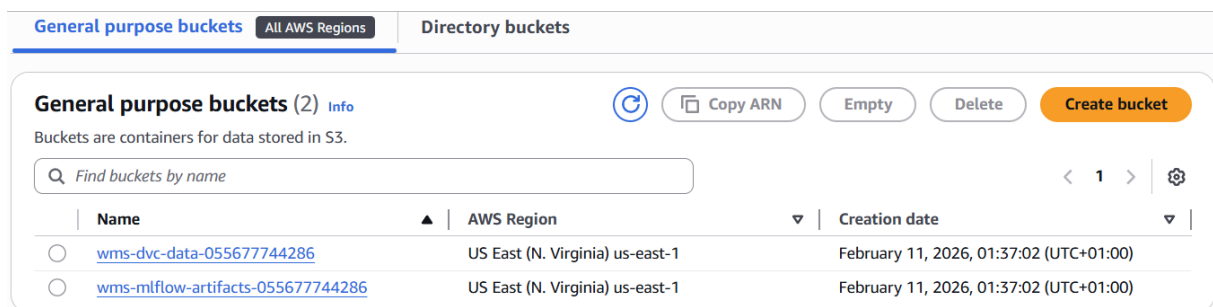
dvc_bucket = "wms-dvc-data-055677744286"
ec2_instance_id = "i-0abe1fe1833dbecde"
ec2_public_ip = "34.199.41.233"
ecr_repository_url = "055677744286.dkr.ecr.us-east-1.amazonaws.com/wms-model"
grafana_port_forward = "kubectl port-forward -n monitoring svc/kube-prometheus-stack-grafana 3000:80"
mlflow_bucket = "wms-mlflow-artifacts-055677744286"
mlflow_url = "http://34.199.41.233:5000"
monitoring_enabled = true
prometheus_port_forward = "kubectl port-forward -n monitoring svc/kube-prometheus-stack-prometheus 9090:9090"
ssh_command = "ssh -i ~/.ssh/vockey.pem ec2-user@34.199.41.233"
ssh_tunnel_grafana = "ssh -i ~/.ssh/vockey.pem -L 3000:localhost:3000 ec2-user@34.199.41.233"
ssh_tunnel_prometheus = "ssh -i ~/.ssh/vockey.pem -L 9090:localhost:9090 ec2-user@34.199.41.233"
```

Rys. 18 Wynik polecenia `terraform apply` — lista 12 utworzonych zasobów AWS

Rysunki 19 i 20 przedstawiają odpowiednio instancję EC2 i zasobniki S3 w konsoli AWS.



Rys. 19 Instancja EC2 wms-k3s w konsoli AWS



Rys. 20 Zasobniki S3: wms-dvc-data-* dla danych treningowych i wms-mlflow-artifacts-* dla artefaktów

6.2 Konfiguracja EC2 i k3s

Konfiguracja instancji EC2 odbywa się automatycznie przy pierwszym uruchomieniu za pomocą skryptu `user-data`. Skrypt jest szablonowany przez Terraform, który przy generowaniu pliku podstawiane są zmienne takie jak adres zasobnika S3 i region AWS, dzięki czemu nie ma żadnych wartości zakodowanych na stałe.

Kroki wykonywane przez `user-data`:

1. Instalacja zależności systemowych: `dnf install -y git curl libicu`.
2. Instalacja k3s — uproszczonego Kubernetes [19].
3. Instalacja MLflow i boto3: `pip3 install --ignore-installed mlflow boto3`.
4. Konfiguracja MLflow jako usługi systemd:

[Service]

```
ExecStart=mlflow server \
  --host 0.0.0.0 --port 5000 \
  --default-artifact-root s3://${mlflow_bucket}/
```

5. Instalacja GitHub Actions runnera jako usługi systemd.

Instancja korzysta z Amazon Linux 2023, który oferuje nowszy OpenSSL 3.0 i wsparcie producenta do 2028 roku. Po zakończeniu `user-data` instancja jest gotowa do przyjęcia zadań z GitHub Actions bez dalszej konfiguracji ręcznej.

Architektura k3s. K3s to uproszczona dystrybucja Kubernetes zawarta w pojedynczym pliku binarnym. W porównaniu do pełnego Kubernetes zastępuje etcd bazą SQLite, usuwa przestarzałe sterowniki i wbudowuje podstawowe komponenty sieciowe. Dzięki temu działa na instancji `t3.large` (8 GB RAM) ze znaczącym zapasem zasobów dla poda z modelem; pełne Kubernetes samo w sobie wymagałoby podobnej ilości RAM.

W projekcie użyty jest klaster jednorazowy (single-node): jeden węzeł pełni jednocześnie rolę control plane i workera. Taka architektura jest wystarczająca dla jednej działającej aplikacji i eliminuje koszty dodatkowych instancji EC2. Wysoka dostępność (HA) nie była wymaganiem. System jest przeznaczony do demonstracji i testowania, nie do środowiska produkcyjnego.

Strategia aktualizacji. Dla Deploymentu zastosowano domyślną strategię `RollingUpdate`: k3s najpierw uruchamia nowy pod, a dopiero po jego gotowości usuwa stary. Oznacza to zerowy przestój (zero-downtime deployment) przy aktualizacji modelu. Działa to jednak tylko wtedy, gdy instancja EC2 jest uruchomiona. W przeciwnym wypadku klaster jest niedostępny.

Rollbacki i wersjonowanie. Rollbacki na poziomie Kubernetes nie są rozbudowane celowo. Mechanizmem wersjonowania modeli jest MLflow, gdzie każda wersja modelu jest zarejestrowana w rejestrze MLflow ze swoimi metrykami. Rollback polega na przestawieniu stadium modelu w MLflow z `Production` na poprzednią wersję i ponownym uruchomieniu `deploy-model.yaml`. Helm zachowuje historię ostatnich wdrożeń (`helm history wms-api`), co umożliwia też cofnięcie konfiguracji Kubernetes bez ponownego treningu.

Rysunek 21 potwierdza poprawne działanie usług `systemd`, a rysunek 22 przedstawia aktualny stan podów klastra k3s.

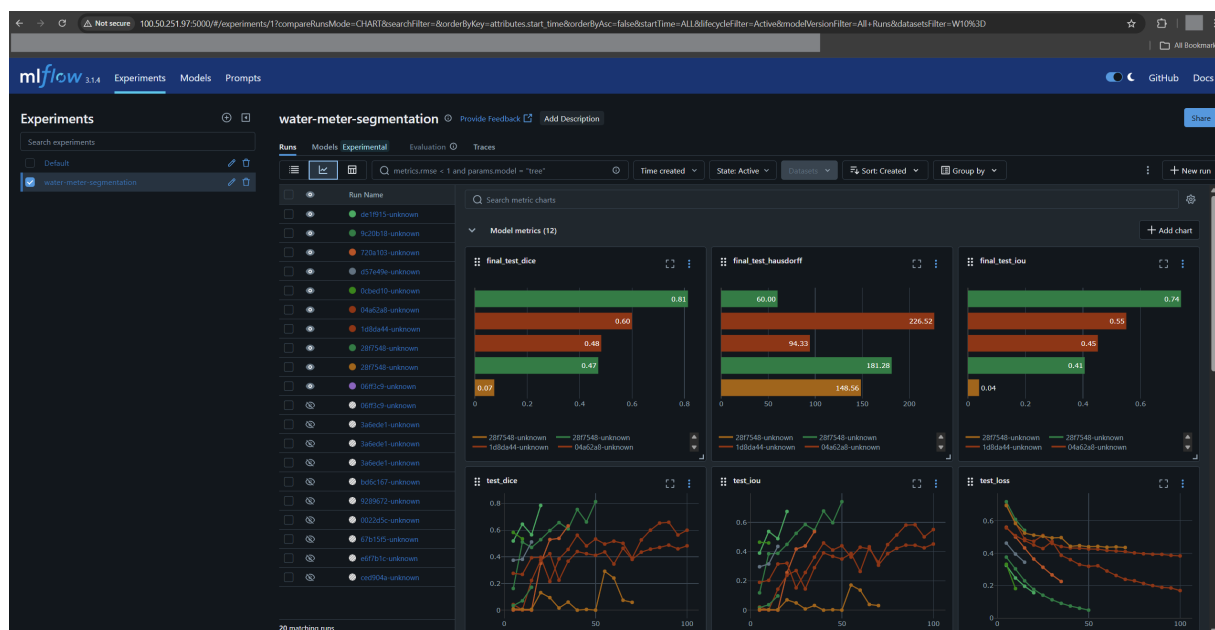
```
ec2-user@ip-10-0-1-221:~$ systemctl status mlflow
mlflow.service - MLflow Tracking Server
Loaded: loaded (/etc/systemd/system/mlflow.service; enabled; preset: disabled)
Active: active (running) since Wed 2026-02-18 00:48:51 UTC; 1h 27min ago
Main PID: 2063 (mlflow)
Tasks: 9 (limit: 9297)
Memory: 481.0M
CPU: 9.265s
Group: /system.slice/mlflow.service
└─ 2063 /usr/bin/python3 /usr/local/bin/mlflow server --backend-store-uri sqlite:///opt/mlflow/mlflow.db --default-artifact-root s3://wms-mlflow-artifacts-055677744286/ --host 0.0.0.0 --workers 2 --gunicorn-opts "--timeout 300 --keep-alive 120"
└─ 2060 /usr/bin/python3 -u gunicorn --timeout 300 --keep-alive 120 -b 0.0.0.0:5000 -w 2 mlflow.server:app
└─ 2410 /usr/bin/python3 -u gunicorn --timeout 300 --keep-alive 120 -b 0.0.0.0:5000 -w 2 mlflow.server:app
└─ 2411 /usr/bin/python3 -u gunicorn --timeout 300 --keep-alive 120 -b 0.0.0.0:5000 -w 2 mlflow.server:app
Feb 18 00:51:59 ip-10-0-1-221.ec2.internal mlflow[2410]: INFO [alembic.runtime.migration] Context impl SQLiteImpl.
Feb 18 00:51:59 ip-10-0-1-221.ec2.internal mlflow[2410]: INFO [alembic.runtime.migration] Will assume non-transactional DDL.
Feb 18 00:51:59 ip-10-0-1-221.ec2.internal mlflow[2410]: INFO [alembic.runtime.migration] Context impl SQLiteImpl.
Feb 18 00:51:59 ip-10-0-1-221.ec2.internal mlflow[2410]: INFO [alembic.runtime.migration] Will assume non-transactional DDL.
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: 2026/02/18 00:52:20 INFO mlflow.store.db.utils: Creating initial mlflow database tables...
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: 2026/02/18 00:52:20 INFO mlflow.store.db.utils: Updating database tables
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: INFO [alembic.runtime.migration] Context impl SQLiteImpl.
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: INFO [alembic.runtime.migration] Will assume non-transactional DDL.
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: INFO [alembic.runtime.migration] Context impl SQLiteImpl.
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: INFO [alembic.runtime.migration] Will assume non-transactional DDL.
ec2-user@ip-10-0-1-221:~$ systemctl status actions.runner
actions.runner.service - GitHub Actions Runner (Rafalost-Water-Meters-Segmentation-Autimatization.ip-10-0-1-221)
Loaded: loaded (/etc/systemd/system/actions.runner.service; enabled; preset: disabled)
Active: active (running) since Wed 2026-02-18 00:48:51 UTC; 1h 26min ago
Main PID: 2054 (runsv.sh)
Tasks: 20 (limit: 9297)
Memory: 153.0M
CPU: 2.878s
Group: /system.slice/actions.runner.Rafalost-Water-Meters-Segmentation-Autimatization.ip-10-0-1-221.service
└─ 2054 /bin/bash /home/ec2-user/actions-runner/runsv.sh
└─ 2060 /external/node20/bin/node /bin/runnerService.js
└─ 2127 /home/ec2-user/actions-runner/bin/runner.Listener run --startuptime service
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal systemd[1]: Started actions.runner.Rafalost-Water-Meters-Segmentation-Autimatization.ip-10-0-1-221.service - GitHub Actions Runner (Rafalost-Water-Meters-Segmentation-Autimatization.ip-10-0-1-221).
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal runsv.sh[2054]: path=/home/ec2-user/.local/bin:/home/ec2-user/bin:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal runsv.sh[2060]: Starting Runner listener with startup type: service
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal runsv.sh[2060]: Started listener process, pid: 2127
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal runsv.sh[2060]: Started running service
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal runsv.sh[2060]: v Connected to GitHub
Feb 18 00:48:54 ip-10-0-1-221.ec2.internal runsv.sh[2060]: Current runner version: '2.311.0'
Feb 18 00:48:54 ip-10-0-1-221.ec2.internal runsv.sh[2060]: 2026-02-18 00:48:54Z: Listening for Jobs
ec2-user@ip-10-0-1-221:~$
```

Rys. 21 Stan usług `mlflow` i `actions.runner` w `systemd` — obie usługi aktywne

```
[ec2-user@ip-10-0-1-221 ~]$ kubectl get pods -A
NAMESPACE      NAME                                     READY   STATUS    RESTARTS   AGE
default         wms-model-m1-model-d74dcb96f-wtn8z    1/1     Running   2 (89m ago)  97m
kube-system     coredns-67d69c9b5b-xrvzt              1/1     Running   39 (94m ago)  5d6h
kube-system     helm-install-traefik-crd-lfp4z         0/1     Completed 0           5d6h
kube-system     helm-install-traefik-mljg7             0/1     Completed 2           5d6h
kube-system     local-path-provisioner-546dfc6456-x6m4t 1/1     Running   40 (94m ago)  5d6h
kube-system     metrics-server-7b9c9c4b9c-52nmg        1/1     Running   39 (94m ago)  5d6h
kube-system     svclb-traefik-3f01b321-p2jcr           2/2     Running   78 (94m ago)  5d6h
kube-system     traefik-55db578d67-p44ld               1/1     Running   39 (94m ago)  5d6h
monitoring      alertmanager-kube-prometheus-stack-alertmanager-0 2/2     Running   16 (94m ago)  45h
monitoring      kube-prometheus-stack-grafana-65c4485b4d-wrqjs 3/3     Running   6 (94m ago)  97m
monitoring      kube-prometheus-stack-kube-state-metrics-8f4c577fd-7bx2d 1/1     Running   8 (94m ago)  45h
monitoring      kube-prometheus-stack-operator-5bd989bd85-49z16 1/1     Running   2 (94m ago)  97m
monitoring      kube-prometheus-stack-prometheus-node-exporter-rckwh 1/1     Running   8 (94m ago)  45h
monitoring      prometheus-kube-prometheus-stack-prometheus-0 2/2     Running   16 (94m ago)  45h
```

Rys. 22 Pody klastra k3s — `kubectl get pods -A`. Widoczny pod API modelu oraz stos monitorowania.

Interfejs MLflow dostępny jest pod adresem <http://100.50.251.97:5000> (rys. 23).



Rys. 23 Interfejs MLflow (<http://100.50.251.97:5000>) — lista eksperymentów i przebiegów treningowych z metrykami

6.3 Dostępne porty i usługi

Security Group Terraform otwiera wyłącznie porty niezbędne do działania systemu. Tabela 2 zestawia wszystkie dostępne punkty dostępu.

Tabela 2 Porty dostępne na instancji EC2 wms-k3s

Port	Usługa	Przeznaczenie
22	SSH	Zarządzanie instancją EC2 (tylko IP operatora)
5000	MLflow	Interfejs śledzenia eksperymentów i rejestr modeli
30080	API modelu (NodePort)	Predykcja segmentacji — endpoint <code>/predict</code>
30300	Grafana (NodePort)	Dashboard wizualizacji metryk
30900	Prometheus (NodePort)	Interfejs zapytań i podgląd <i>Targets</i>

Ruch przychodzący na porty 5000, 30080, 30300 i 30900 jest dozwolony z dowolnego ad-

resu IP (0.0.0.0/0), ponieważ EC2 nie jest wyposażone w Load Balancer z certyfikatem TLS, a połączenia odbywają się po HTTP. Port SSH (22) jest ograniczony do konkretnego adresu IP operatora skonfigurowanego w terraform.tfvars. Konfigurację Security Group przedstawia rysunek 24.

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-0f544380b4c910e65	IPv4	Custom TCP	TCP	30900	46.112.114.36/32	Prometheus
-	sgr-0201321fd5a004a2f	IPv4	Custom TCP	TCP	30080	0.0.0.0/0	The application
-	sgr-099bda2b9f4c6cf71	IPv4	SSH	TCP	22	46.112.114.36/32	SSH from my IP (auto-d...
-	sgr-00c3fe3405d3170b3	IPv4	Custom TCP	TCP	30300	46.112.114.36/32	Grafana
-	sgr-00847d17feeffcSaa	IPv4	Custom TCP	TCP	8000	0.0.0.0/0	HTTP from my IP (auto...
-	sgr-05738bf1210f436ee	IPv4	Custom TCP	TCP	5000	0.0.0.0/0	MLflow from anywhere...

Rys. 24 Security Group instancji EC2 — reguły ruchu przychodzącego z otwartymi portami usług

6.4 Konfiguracja Helm

W projekcie Helm [7] wykorzystywany jest dwójako: do wdrożenia własnego charta API modelu oraz do konfiguracji zewnętrznego stosu monitorowania.

Chart API modelu. Katalog `devops/helm/ml-model/` zawiera własny chart Kubernetes dla serwera API. Zamiast zarządzać oddzielnymi plikami manifestów, całą aplikację opisuje jeden zestaw szablonów YAML sparametryzowanych zmiennymi. Bez Helm każda aktualizacja wymagałaby ręcznej edycji kilku plików i kolejnych wywołań `kubectl apply`. Helm sprowadza to do jednej komendy:

```
helm upgrade --install wms-model devops/helm/ml-model/ \
  -f infrastructure/helm-values.yaml \
  --namespace default --create-namespace
```

Komenda ta działa zarówno przy pierwszym wdrożeniu (*install*) jak i przy aktualizacji (*upgrade*); Helm sam sprawdza czy release już istnieje. Helm zachowuje historię wdrożeń (`helm history wms-model`), co umożliwia szybki rollback do poprzedniej wersji.

Struktura charta:

```
devops/helm/ml-model/
+-- Chart.yaml          <- metadane (nazwa, wersja charta)
+-- values.yaml         <- wartosci domyslne (NodePort 30080, limity RAM/CPU)
|-- templates/
  +-- _helpers.tpl      <- funkcje pomocnicze Helm
  +-- deployment.yaml   <- zasob Deployment
  +-- service.yaml      <- zasob Service (NodePort 30080)
  -- servicemonitor.yaml <- ServiceMonitor dla Prometheus
```

Szablony korzystają ze składni Go Templates i odwołują się do wartości przez `.Values.*`. Plik `infrastructure/helm-values.yaml` nadpisuje wartości domyślne charta dla konkretnego wdrożenia. Najważniejsze fragmenty:

```

image:
  repository: "055677744286.dkr.ecr.us-east-1.amazonaws.com/wms-model"
  pullPolicy: Always
  tag: "latest"

imagePullSecrets:
  - name: ecr-secret      # dane do logowania do ECR

env:
  - name: MLFLOW_TRACKING_URI
    value: "http://localhost:5000" # MLflow na hoscie EC2 (hostNetwork)
  - name: MODEL_VERSION
    value: "production"

resources:
  limits:
    memory: "1Gi"
    cpu: "500m"
  requests:
    memory: "512Mi"
    cpu: "100m"

```

Warto zwrócić uwagę na `MLFLOW_TRACKING_URI: localhost:5000`; kontener korzysta z `hostNetwork: true`, co pozwala mu odwoływać się do MLflow działającego bezpośrednio na maszynie EC2, bez konieczności dodatkowej usługi Kubernetes. Przy każdym wdrożeniu GitHub Actions podaje aktualny hash commita jako `-set image.tag=${COMMIT_SHA}`, co zapewnia jednoznaczną identyfikację wersji obrazu.

Postęp wdrożenia API można śledzić w czasie rzeczywistym poleceniem:

```
kubectl rollout status deployment/wms-model-ml-model --watch
```

Po wdrożeniu API jest dostępne pod adresem `http://100.50.251.97:30080`.

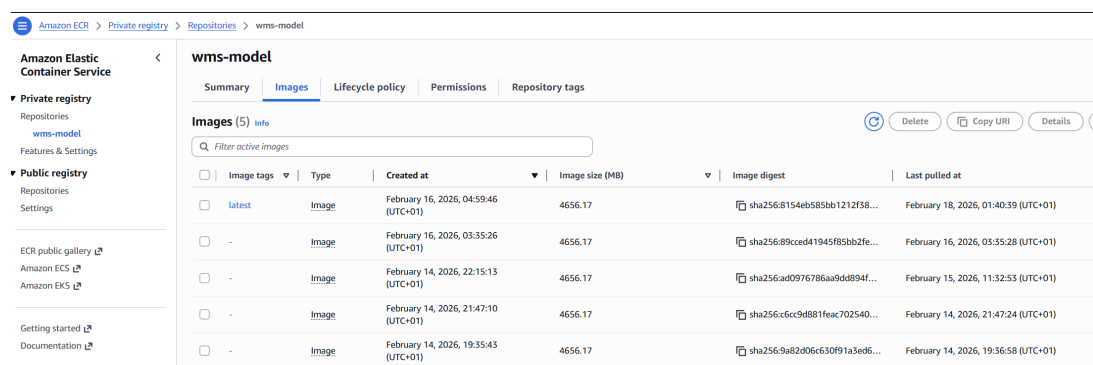


Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
latest	Image	February 16, 2026, 04:59:46 (UTC+01)	4656.17	sha256:8154eb585bb1212f58...	February 18, 2026, 01:40:39 (UTC+01)
-	Image	February 16, 2026, 03:35:26 (UTC+01)	4656.17	sha256:89cccd41945f85bb2fe...	February 16, 2026, 03:35:28 (UTC+01)
-	Image	February 14, 2026, 22:15:13 (UTC+01)	4656.17	sha256:ad0976786aa94d894f...	February 15, 2026, 11:32:53 (UTC+01)
-	Image	February 14, 2026, 21:47:10 (UTC+01)	4656.17	sha256:c6cc9d881feac702540...	February 14, 2026, 21:47:24 (UTC+01)
-	Image	February 14, 2026, 19:35:43 (UTC+01)	4656.17	sha256:9a82d06c630f91a3ed6...	February 14, 2026, 19:36:58 (UTC+01)

Rys. 25 Repozytorium `wms-model` w Amazon ECR z obrazami oznaczonymi hashem commita

Dostępne endpointy API to `/health` (status usługi), `/predict` (predykcja segmentacji) i

/metrics (metryki Prometheus). Odpowiedź endpointu /health przedstawia rysunek 26a, a pełną listę eksponowanych metryk prezentuje rysunek 26b.

```

{"status": "healthy", "model_loaded": true, "device": "cpu"}

```

(a) Odpowiedź endpointu /health wdrożonego API modelu

```

# HELP wms_predict_latency_seconds_created Prediction latency in seconds
# TYPE wms_predict_latency_seconds_created gauge
wms_predict_latency_seconds_created 1.7713757733752115e+09
# HELP wms_predict_errors_total Total number of prediction errors
# TYPE wms_predict_errors_total counter
wms_predict_errors_total 0.0
# HELP wms_predict_errors_created Total number of prediction errors
# TYPE wms_predict_errors_created gauge
wms_predict_errors_created 1.7713757733752952e+09
# HELP wms_model_loaded Model loaded status (1=loaded, 0=not loaded)
# TYPE wms_model_loaded gauge
wms_model_loaded 1.0

```

(b) Metryki Prometheus eksponowane pod adresem /metrics: liczba predykcji, czasy odpowiedzi i błędy

Rys. 26 Endpointy diagnostyczne API modelu

6.5 Stos monitorowania

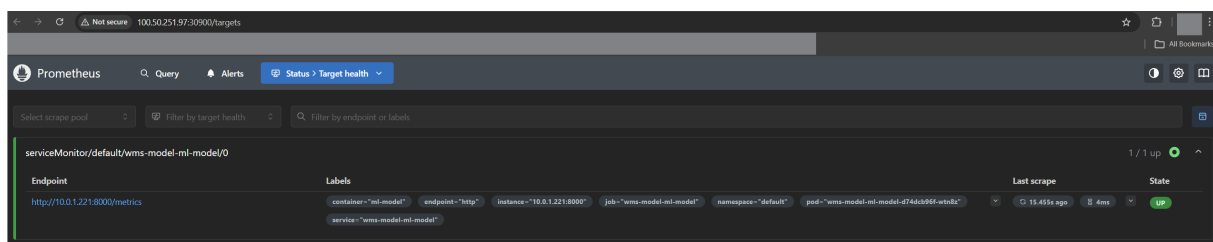
Monitoring oparty na Prometheus [18] i Grafana wdrażany jest jako oddzielny stos na tym samym klastrze k3s. Składa się z trzech komponentów:

- **Prometheus** — zbiera metryki z endpointu /metrics API modelu co 15 sekund; konfiguracja scraping w prometheus.yml,
- **Grafana** — wizualizuje metryki na predefiniowanych dashboardach; dostępna na porcie NodePort 30300,
- **kube-state-metrics** — dostarcza metryki Kubernetes (stan podów, zużycie zasobów węzła).

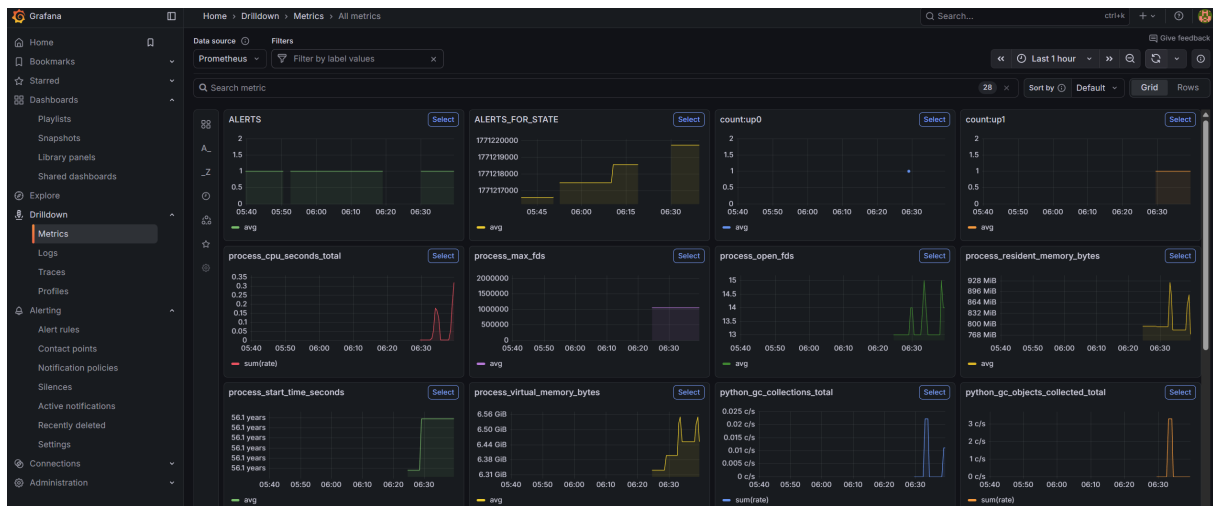
API modelu eksponuje metryki w formacie Prometheus przy użyciu biblioteki prometheus_client. Śledzone metryki to:

- wms_predictions_total — łączna liczba żądań predykcji (Counter),
- wms_predict_latency_seconds — histogram czasów odpowiedzi,
- wms_predict_errors_total — liczba błędów (Counter),
- wms_model_loaded — czy model jest załadowany (Gauge, wartość 0 lub 1).

Działanie stosu monitorowania ilustruje dashboard Grafana (rys. 28) oraz widok Targets w Prometheus (rys. 27).



Rys. 27 Strona Targets w Prometheus (:30900) — endpoint /metrics API w stanie UP



Rys. 28 Dashboard Grafana (<http://100.50.251.97:30300>) — metryki działającego API modelu

6.6 Workflow użytkownika

6.6.1 Scenariusz 1: Wdrożenie przez automatyczny pipeline

Jak opisano w rozdziale 5, dostarczenie nowych danych treningowych przez `git push` wyzwała pełny pipeline. Jeśli wytrenowany model przeszedł bramkę jakości, pipeline automatycznie wdraża go na klaster k3s przez `deploy-model.yaml`, a następnie zatrzymuje instancję EC2. Po zakończeniu pipeline'u aplikacja jest zainstalowana na klastrze i gotowa do działania. Aby skorzystać z API, wystarczy uruchomić instancję EC2 za pomocą workflow *Manual EC2 Control* (zakładka *Actions* w GitHub, akcja *start*). Po uruchomieniu workflow wyświetla adres IP w podsumowaniu, a endpoint `/predict` jest natychmiast dostępny.

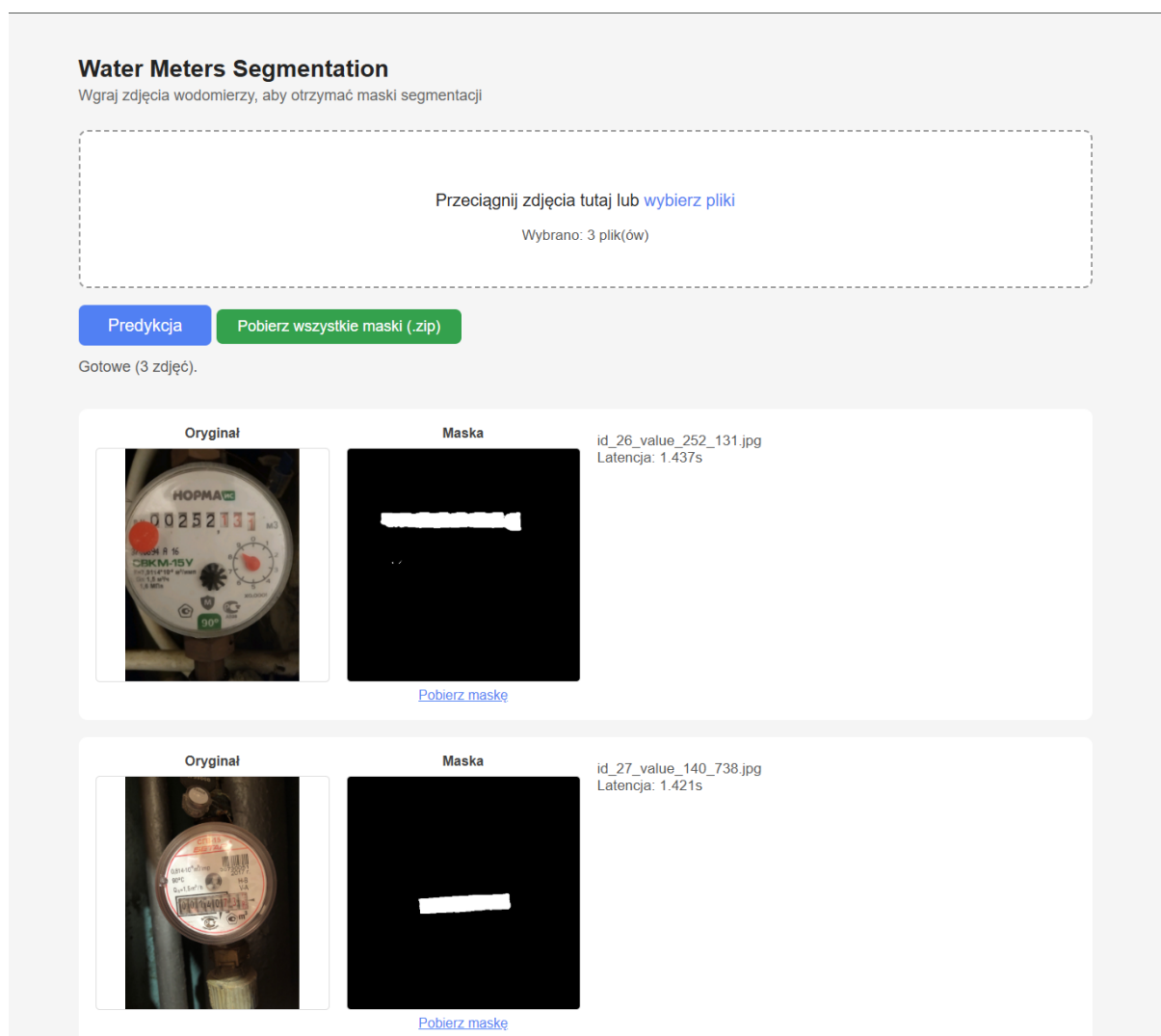
Należy zaznaczyć, że wdrożenie następuje tylko gdy model poprawił metryki. Jeśli trening nie przeszedł bramki jakości, klaster k3s nadal serwuje poprzednią wersję. Do kontrolowanego wdrożenia niezależnego od treningu służy Scenariusz 2.

6.6.2 Scenariusz 2: Ręczne wdrożenie i korzystanie z API (zalecany)

Jest to zalecany sposób korzystania z systemu dla operatora chcącego uruchomić lub zaktualizować działające API. Cały proces odbywa się przez interfejs GitHub Actions i nie wymaga żadnych lokalnych poświadczeń AWS.

Krok 1: Wdrożenie modelu. W zakładce *Actions* należy uruchomić workflow *Release & Deploy* (`release-deploy.yaml`) klikając *Run workflow*. Workflow automatycznie uruchamia instancję EC2, buduje obraz Docker, wypycha go do ECR, pobiera najnowszy model ze stadium Production z MLflow, aktualizuje klaster k3s przez Helm, a po weryfikacji zatrzymuje EC2.

Krok 2: Uruchomienie instancji do predykcji. Po wdrożeniu model jest zainstalowany na klastrze, ale EC2 jest zatrzymany w celu oszczędności. Aby wysłać zapytania do API, należy uruchomić instancję workflow *Manual EC2 Control* z akcją *start*. Workflow wyświetla aktualne IP i URL MLflow w podsumowaniu uruchomienia. API jest dostępne pod adresem `http://100.50.251.97:30080/`.



Rys. 29 Działający interfejs webowy API modelu: segmentacja trzech zdjęć wodomierzy z wygenerowanymi maskami i zmierzoną latencją predykcji

Po zakończeniu pracy należy ponownie uruchomić *Manual EC2 Control* z akcją *stop*, aby nie generować zbędnych kosztów.

6.6.3 Scenariusz 3: Lokalna predykcja (wyłącznie dla deweloperów)

Predykcję można uruchomić lokalnie bez infrastruktury chmurowej. Opcja ta wymaga jednak spełnienia warunków niedostępnych dla typowego użytkownika końcowego.

Wymagania wstępne:

- działająca instancja EC2 z MLflow (model musi być zarejestrowany w stadium Production),
- lokalne poświadczenia AWS skonfigurowane w `~/.aws/credentials:AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN,`
- Python z zależnościami projektu zainstalowanymi lokalnie.

Przy spełnieniu powyższych warunków predykcja przebiega następująco:

1. Pobranie modelu Production z MLflow na dysk lokalny:

```
python WMS/scripts/sync_model_aws.py
```

2. Umieszczenie obrazów do segmentacji w `WMS/data/predict/input/`.
3. Uruchomienie predykcji: `python WMS/src/predicts.py`
4. Wyniki zapisywane są w `WMS/data/predict/output/`.

Poświadczenia AWS w środowisku AWS Academy są tymczasowe (ważność 4 godziny) i wymagają ręcznego odnawiania przy każdej sesji laboratoryjnej. Z tego powodu opcja lokalnej predykcji jest praktycznie dostępna wyłącznie dla operatora posiadającego bezpośredni dostęp do konta AWS, a nie dla użytkownika końcowego.

Opisane scenariusze pokrywają wszystkie typowe przypadki interakcji z systemem.

6.7 Dostępne operacje zarządzania wdrożeniem

System udostępnia cztery workflow GitHub Actions do zarządzania infrastrukturą i wdrożeniami. Tabela 3 zestawia je wraz z wyzwalaczami i przeznaczeniem.

Tabela 3 Workflow GitHub Actions dostępne w systemie

Workflow	Wyzwalacz	Przeznaczenie
training-data-pipeline	Push do data/*, workflow_dispatch	Pełny pipeline: pobieranie danych z S3, walidacja, trening, bramka jakości. Jeśli model poprawił metryki: wdrożenie na k3s i scalenie PR. EC2 uruchamiany i zatrzymywany automatycznie.
release-deploy	Push do main (zmiany kodu/Docker/Helm), workflow_dispatch	Wdrożenie modelu na klaster k3s bez treningu: build Docker, push do ECR, aktualizacja Helm. Używany przy zmianach kodu serwowania lub jako ręczne ponowne wdrożenie. EC2 uruchamiany i zatrzymywany automatycznie.
ec2-manual-control	Tylko workflow_dispatch	Ręczne uruchomienie lub zatrzymanie instancji EC2. Po starcie wyświetla aktualne IP i URL MLflow w podsumowaniu. Używany gdy potrzebny jest dostęp do API lub interfejsu MLflow.
deploy-model	Reużywalny (workflow_call), workflow_dispatch	Właściwy krok wdrożenia wywołany przez training-data-pipeline i release-deploy. Można też uruchomić samodzielnie. Działa na self-hosted runnerze EC2, uwierzytelniając się przez rolę IAM instancji.

Żaden z powyższych workflow nie wymaga lokalnych poświadczeń AWS na komputerze operatora. Workflow uruchamiane na runnerze `ubuntu-latest` (start/stop EC2) korzystają z sekretów repozytorium GitHub, natomiast `deploy-model` działa na self-hosted runnerze EC2 i uwierzytelnia się przez rolę IAM przypisaną do instancji.

7 Analiza Wyników i Działania Systemu

7.1 Metodologia porównania

Celem eksperymentów jest ilościowe i jakościowe porównanie dwóch podejść do zarządzania cyklem życia modelu uczenia maszynowego [11]:

1. **Podejście ręczne** — wszystkie kroki (przygotowanie danych, trening, walidacja, wdrożenie) wykonywane przez człowieka bez automatyzacji.
2. **Zautomatyzowane CI/CD (niniejszy projekt)** — pełna automatyzacja od push danych do wdrożenia modelu w oparciu o GitHub Actions, DVC i MLflow.

Porównanie obejmuje następujące wymiary:

- czas nakładu pracy człowieka w każdym podejściu,
- jakość modelu (metryki Dice i IoU),
- pracochłonność (liczba ręcznych kroków),
- ryzyko błędu ludzkiego,
- skalowalność procesu.

7.2 Czasy wykonania scenariuszy

Tabela 4 zestawia szacowane czasy nakładu pracy człowieka dla każdego podejścia w scenariuszu dodania nowej partii danych treningowych i wdrożenia ulepszanego modelu. Wartości dla podejścia ręcznego mają charakter szacunkowy i zakładają doświadczonego inżyniera ML znającego narzędzia projektu. Wartości dla podejścia zautomatyzowanego zostały zmierzone na podstawie logów GitHub Actions.

Tabela 4 Porównanie nakładu pracy człowieka w obu scenariuszach

Etap	Ręczny	Zautomatyzowane CI/CD
Przygotowanie i upload danych	~30 min	~10 min
Walidacja danych	~30 min	~10 min (poprawki złych danych)
Trening modelu	~3 h	automatyczny
Ocena jakości	~15 min	automatyczna
Wdrożenie	~45 min	automatyczne
Nakład człowieka łącznie	~5 h	~20 min
Całkowity czas pipeline'u	~5 h	~4 h

Kluczową obserwacją jest rozróżnienie między *nakładem pracy człowieka* a *całkowitym czasem realizacji*. W podejściu ręcznym są one tożsame, ponieważ człowiek jest zaangażowany przez cały czas trwania procesu. W podejściu zautomatyzowanym inżynier wykonuje jedynie komendę `git push` (poniżej jednej minuty), po czym pipeline działa bez jego udziału. Całkowity czas działania pipeline'u, od push do wdrożenia, można odczytać z zakładki *Actions* w repozytorium GitHub.

7.3 Metryki modelu

Tabela 5 przedstawia metryki modelu dla kolejnych wersji wytrenowanych w ramach projektu. Baseline pochodzi z repozytorium referencyjnego *Water-Meters-Segmentation* [10]. Metryki są rejestrowane automatycznie przez MLflow [15] podczas każdego treningu.

Tabela 5 Metryki jakości modelu dla kolejnych wersji

Wersja modelu	Dice	IoU	Hausdorff
v21	0,8067	0,7286	49,0034
v20	0,8134	0,7409	59,9987
v18	0,5998	0,5508	226,5247
v16	0,4825	0,4518	94,3321
v14	0,4710	0,4060	181,2840
v13	0,0741	0,0388	148,5599
v12	0,0042	0,0021	358,6141
v11	0,0449	0,0233	355,3105
v10	0,0449	0,0233	355,3105
v9	0,3116	0,2461	329,8586
v8	0,0536	0,0278	336,6613
v7	0,0536	0,0278	336,6613
v5	0,3125	0,1852	272,5949
v3	$3,5689 \cdot 10^{-10}$	$3,5689 \cdot 10^{-10}$	724,0773
v1	$3,5386 \cdot 10^{-10}$	$3,5386 \cdot 10^{-10}$	409,2163

Rysunek 30 wizualizuje te same wartości w interfejsie MLflow, potwierdzając ich poprawne zarejestrowanie po każdym przebiegu treningowym.



Rys. 30 Metryki jakości modelu w interfejsie MLflow

Spośród wszystkich wersji jedynie te spełniające jednocześnie oba kryteria bramki jakości — wyższy Dice i IoU od aktualnego modelu Production — zostały awansowane do stadium Production. Tabela 6 zestawia wyłącznie te wersje.

Tabela 6 Metryki jakości modelu typu production dla kolejnych wersji

Wersja modelu	Dice	IoU	Hausdorff
v20	0,8134	0,7409	59,9987
v18	0,5998	0,5508	226,5247
v16	0,4825	0,4518	94,3321
v14	0,4710	0,4060	181,2840
v9	0,3116	0,2461	329,8586
v5	0,3125	0,1852	272,5949
v3	$3,5689 \cdot 10^{-10}$	$3,5689 \cdot 10^{-10}$	724,0773
v1	$3,5386 \cdot 10^{-10}$	$3,5386 \cdot 10^{-10}$	409,2163

Rysunek 31 przedstawia widok rejestru MLflow z wyłącznie tymi wersjami, skąd API pobie-

rało model przy każdym starcie kontenera.



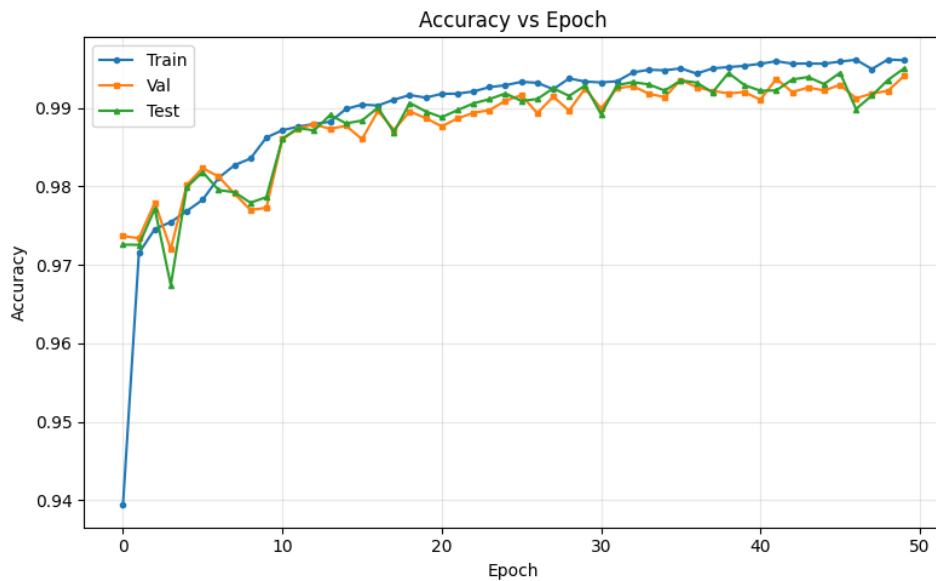
Rys. 31 Metryki jakości modelu w interfejsie MLflow który miały stage Production

Metryki można odczytać z interfejsu MLflow w zakładce *Models* po wybraniu wersji oznaczonej stage *Production*. Bramka jakości w pipeline automatycznie porównuje nowy model z aktualną wersją produkcyjną i tworzy pull request tylko w przypadku jednoczesnej poprawy obu metryk.

Warto zauważyć, że wczesne wersje modelu (v1, v3) uzyskały wartości Dice i IoU rzędu 10^{-10} , co w praktyce oznacza brak jakiejkolwiek zdolności predykcyjnej. Tak skrajnie niskie wartości nie są widoczne na wykresach interfejsu MLflow, który ustala skalę do wartości znaczących (z obsługa zera jako wyjątku). Jest to ograniczenie warstwy wizualizacyjnej frameworka, nie błąd eksperymentu ani rejestracji metryk. Same wartości zostały poprawnie zalogowane i są dostępne w widoku tabelarycznym.

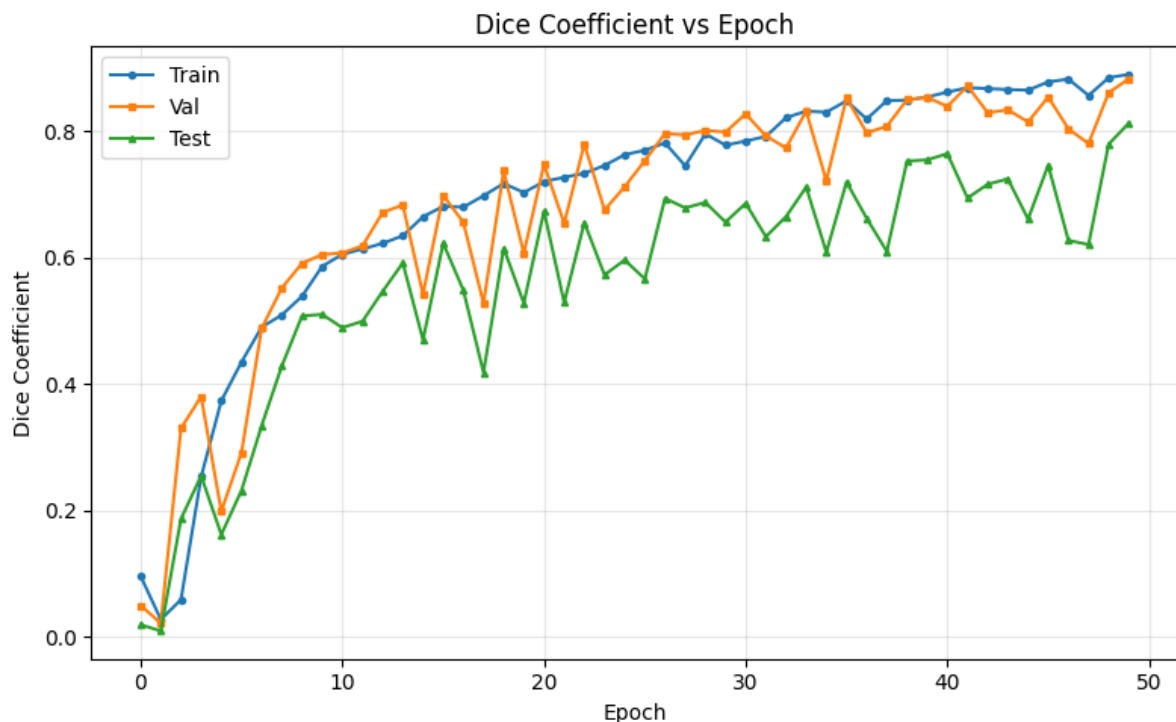
Analiza zbioru wersji potwierdza prawidłowe działanie mechanizmów wersjonowania i kontroli jakości. Wszystkie wersje modelu były rejestrowane w MLflow niezależnie od wyniku, co umożliwia pełną historię eksperymentów. Do etapu *Production* awansowały wyłącznie wersje spełniające wymagania bramki jakości, a tylko te wersje były następnie pobierane przez serwis inferencyjny i udostępniane użytkownikom. Pozostałe wersje zachowały stage *None* i nie uczestniczyły w deploymencie.

Rysunki 32–35 ilustrują przebieg treningu najlepszego modelu (v20): krzywe Accuracy, Dice, IoU oraz funkcji straty rejestrowane w MLflow co pięć epok. Zbieżność krzywych treningowej i walidacyjnej we wszystkich czterech przypadkach wskazuje na brak przeuczenia modelu w toku całego procesu uczenia.



Rys. 32 Wykres metryki Accuracy dla najlepszej wersji modelu (v20) w interfejsie MLflow

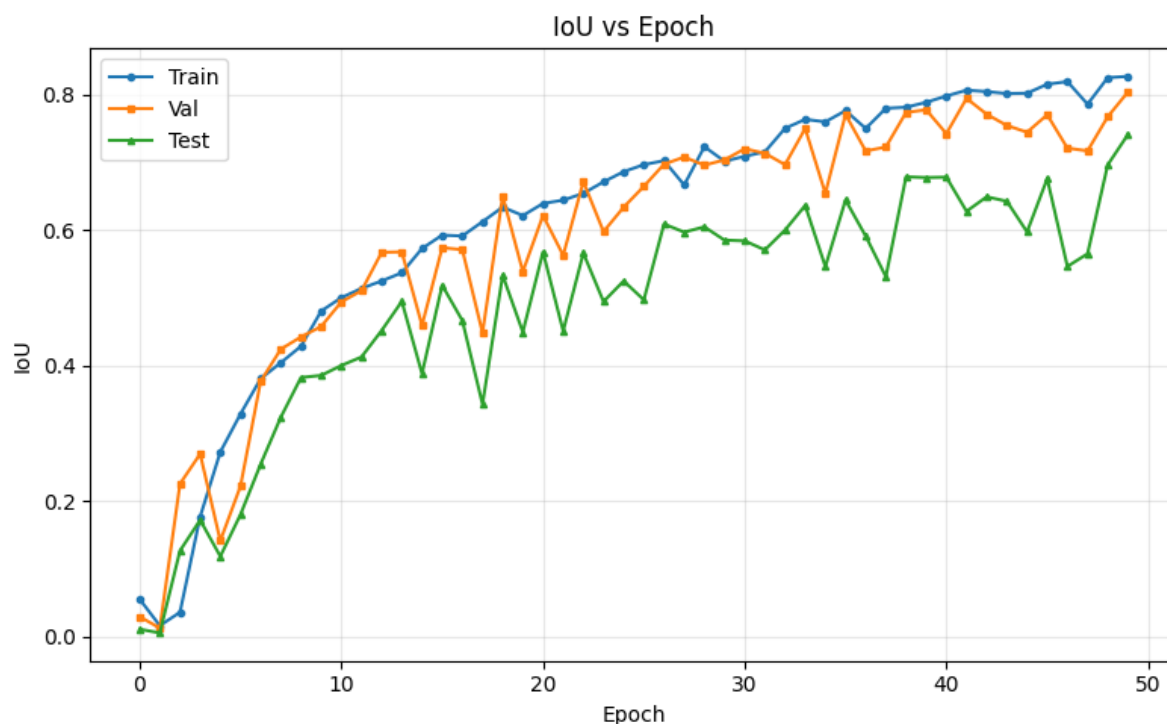
Metryka Accuracy jest miarą ogólną i osiąga wyższe wartości niż Dice, ponieważ uwzględnia piksele tła jako poprawne predykcje. Krzywa Dice (rysunek 33) bezpośrednio mierzy nakładanie się predykowanej maski z referencyjną i stanowi podstawową metrykę oceny segmentacji — jej wartość końcowa wyniosła 0,8134.



Rys. 33 Wykres metryki Dice dla najlepszej wersji modelu (v20) w interfejsie MLflow

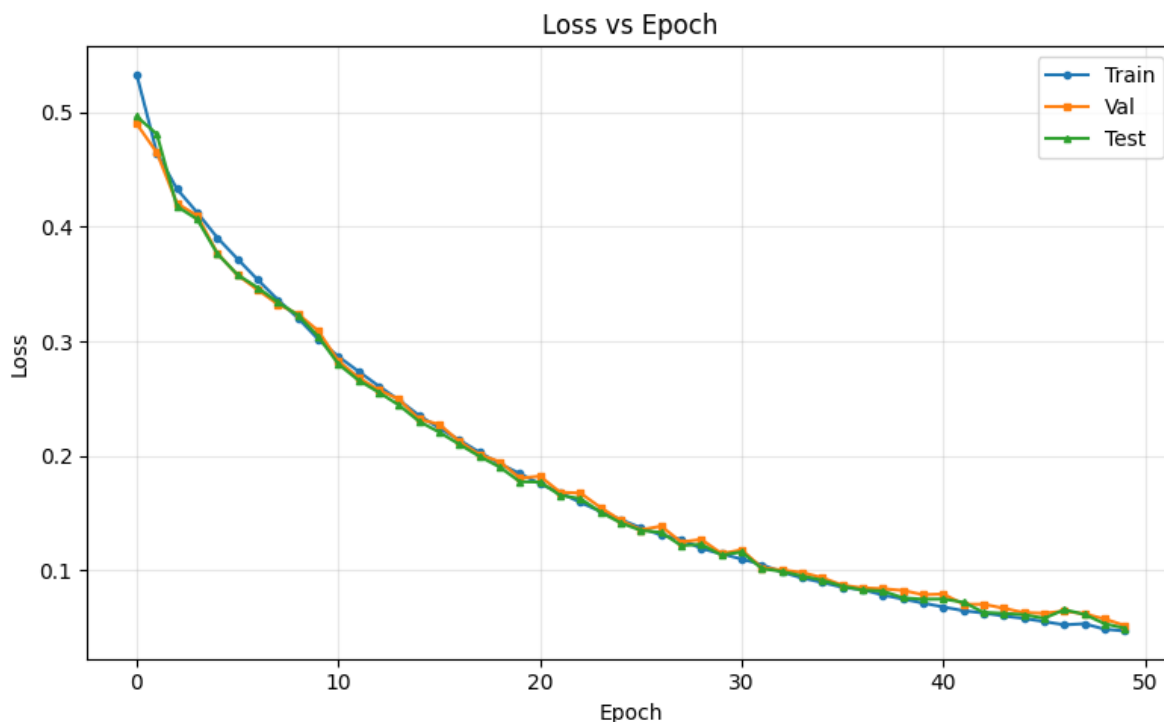
Metryka IoU (rysunek 34), znana również jako Jaccard Index, jest ściśle powiązana z Dice i mierzy stosunek przekroju do sumy predykowanego i referencyjnego obszaru. Dla modelu

v20 osiągnęła wartość końcową 0,7409, co oznacza, że ponad 74% predykowanego obszaru pokrywa się z maską referencyjną.



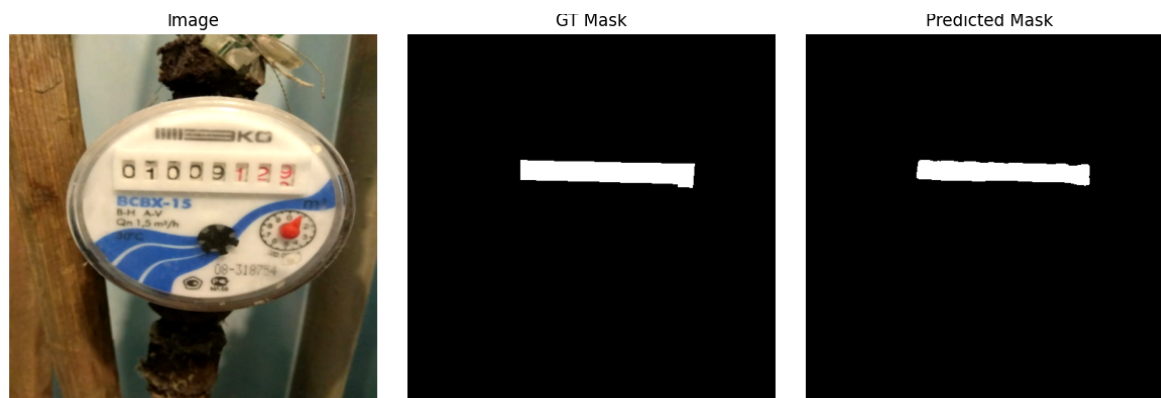
Rys. 34 Wykres metryki IoU dla najlepszej wersji modelu (v20) w interfejsie MLflow

Przebieg funkcji straty (rysunek 35) potwierdza stabilną zbieżność: wartość *validation loss* systematycznie malała przez cały czas treningu bez wyraźnych oznak rozbieżności między zbiorem treningowym a walidacyjnym.



Rys. 35 Wykres funkcji straty dla najlepszej wersji modelu (v20) w interfejsie MLflow

Jakościową ocenę predykcji ilustruje rysunek 36, który zestawia obraz wejściowy, maskę referencyjną (ground truth) i maskę wygenerowaną przez model v20. Kształt i lokalizacja predykowanego obszaru wodomierza są zbliżone do maski referencyjnej, co koresponduje z uzyskanymi wartościami metryk ilościowych.



Rys. 36 Przykład predykcji modelu (v20) w interfejsie MLflow — obraz wejściowy, maska referencyjna i maska przewidywana

7.4 Koszty infrastruktury AWS

Tabela 7 zestawia szacowane koszty poszczególnych zasobów AWS poniesionych w toku realizacji projektu. Dominującą pozycją jest instancja EC2 uruchamiana na czas treningu; pozostałe zasoby (S3, ECR) generują koszty rzędu centów miesięcznie.

Tabela 7 Szacowane koszty infrastruktury AWS

Zasób	Koszt	Uwagi
EC2 t3.large (trening i serwowanie)	\$0,0832/h	Uruchamiana tylko podczas treningu (stop/start)
S3 (artefakty DVC i MLflow)	~\$0,02/mies.	Kilka GB artefaktów
ECR (obrazy Docker)	~\$0,05/mies.	Jeden obraz ≈500 MB
Łącznie (projekt)	20 USD	Za cały okres realizacji

Efektywność kosztową zapewnia architektura stop/start instancji EC2: instancja uruchamiana jest wyłącznie na czas treningu (job `start-infra`) i zatrzymywana bezpośrednio po jego zakończeniu (job `stop-infra`, uruchamiany zawsze niezależnie od wyniku). Rzeczywisty koszt całkowity projektu dostępny jest w konsoli AWS Academy w sekcji *Billing > Bills*.

7.5 Ocena jakościowa

7.5.1 Łatwość wdrożenia

Konfiguracja systemu wymagała jednorazowego nakładu pracy obejmującego: przygotowanie infrastruktury Terraform (VPC, EC2, S3, ECR), konfigurację k3s i Helm, integrację MLflow z S3, napisanie skryptów walidacji danych i bramki jakości oraz przygotowanie workflow GitHub Actions. Jednorazowy koszt konfiguracji wyniósł kilka tygodni pracy. Po jego poniesieniu każde kolejne wdrożenie nowej wersji modelu wymaga jedynie wykonania `git push`.

Podejście ręczne nie wymaga nakładu konfiguracyjnego, ale generuje cykliczny nakład pracy przy każdym wdrożeniu. Przy założeniu jednego wdrożenia miesięcznie i nakładzie ~2,5 h na cykl, automatyzacja staje się opłacalna po kilkunastu cyklach, nie uwzględniając ryzyka błędu ludzkiego, które rośnie z liczbą powtórzeń.

7.5.2 Stabilność systemu

W trakcie testów zaobserwowano następujące zachowania systemu w sytuacjach wyjątkowych:

- **Niepoprawne dane** — walidator odrzucał dane z komunikatem i nie uruchamiał treningu,
- **Model gorszy od baseline** — bramka jakości odrzucała model bez tworzenia PR,
- **Awaria instancji EC2** — job `stop-infra` uruchamiał się zawsze (`if: always()`), zapobiegając wyciekom kosztów.

7.5.3 Skalowalność

Zastosowane podejście ma następujące ograniczenia skalowalności:

- single-node k3s — brak wysokiej dostępności,
- jeden runner EC2 — brak równoległych treningów,
- model proof of concept — nie testowano przy zbiorach > 1000 obrazów.

8 Podsumowanie i wnioski

8.1 Efektywność automatyzacji

Celem niniejszej pracy było zaprojektowanie, wdrożenie i ocena zautomatyzowanego pipeline CI/CD dla modeli uczenia maszynowego, a następnie porównanie tego podejścia z tradycyjnym, ręcznym procesem ponownego uczenia i wdrażania modeli AI. Zrealizowany system objął pełny cykl życia modelu: od dostarczenia nowych danych treningowych za pośrednictwem polecenia `git push`, przez walidację danych, trening i ocenę jakości, aż po wdrożenie modelu w postaci serwisu REST API dostępnego dla użytkowników zewnętrznych. W toku eksperymentów model U-Net przeszedł przez sześć iteracji doskonalenia sterowanych pipeline'em, osiągając współczynnik Dice równy 0,81 oraz IoU równy 0,74, przy czym każda iteracja była inicjowana wyłącznie przez dostarczenie nowych danych treningowych, bez jakiegokolwiek ręcznej ingerencji w przebieg procesu.

Wyniki przeprowadzonego eksperymentu ilościowego potwierdzają przewagę podejścia zautomatyzowanego w zakresie efektywności operacyjnej. Nakład pracy człowieka uległ redukcji z około 5 godzin w podejściu manualnym do poniżej jednej minuty na cykl wdrożenia, przy koszcie obliczeniowym rzędu \$0,29 za jedną sesję treningową na instancji EC2 `t3.large`. Łączny koszt realizacji projektu wyniósł około \$20 przy zakładanym budżecie \$50, co stanowi potwierdzenie skuteczności zastosowanej architektury stop/start, w której zasoby obliczeniowe uruchamiane są wyłącznie na czas treningu.

8.2 Porównanie mocnych i słabych stron obu podejść

Podejście zautomatyzowane eliminuje szereg ryzyk nieodłącznie związanych z procesem manualnym. Dynamiczna bramka jakości, porównująca nowy model z aktualną wersją produkcyjną zarejestrowaną w MLflow, wyklucza możliwość awansowania modelu o niższej jakości zarówno wskutek przeoczenia, jak i pod wpływem presji operacyjnej. Wersjonowanie danych za pomocą DVC [9] w połączeniu z rejestrem modeli MLflow zapewnia pełną odtwarzalność eksperymentów: dla każdej wersji modelu możliwe jest precyzyjne wskazanie zbioru danych oraz parametrów użytych podczas treningu, co w podejściu manualnym nie może być zagwarantowane w sposób systematyczny. Istotną właściwością systemu jest również skalowalność: wzrost rozmiaru zbioru danych nie generuje dodatkowego nakładu pracy inżyniera, gdyż całość obciążenia obliczeniowego przejmuje infrastruktura chmurowa.

Podejście manualne zachowuje natomiast pewne zalety. Nie wymaga ponoszenia jednorazowego kosztu konfiguracji, który w przypadku niniejszego projektu wyniósł kilka tygodni pracy, oraz cechuje się większą elastycznością w sytuacjach nieprzewidzianych przez zdefiniowany pipeline. Pozostaje ono uzasadnioną opcją w wąskich scenariuszach: przy bardzo rzadkich aktualizacjach modelu, małym zbiorze danych oraz przy braku wymagań dotyczących powtarzalności lub audytowalności procesu. Poza tym scenariuszem koszt wdrożenia automatyzacji zwraca się po kilkunastu cyklach, a każde kolejne uruchomienie generuje oszczędność rzędu kilku godzin pracy inżyniera [8].

Słabą stroną zrealizowanego systemu, wynikającą ze świadomych decyzji projektowych podyktowanych ograniczeniami środowiska AWS Academy, jest architektura oparta na poje-

dynczym węźle: klaster k3s uruchomiony na jednej instancji EC2 nie zapewnia wysokiej dostępności, a awaria tej instancji skutkuje niedostępnością zarówno API modelu, jak i rejestru MLflow. Ograniczenia środowiska laboratoryjnego wymusiły ponadto ręczną rotację tymczasowych kluczy dostępowych AWS przy każdym odnowieniu sesji laboratoryjnej oraz uniemożliwiły wdrożenie federacji tożsamości OIDC dla GitHub Actions. W środowisku produkcyjnym obydwie te problemy zostałyby wyeliminowane przez standardowe mechanizmy zarządzania tożsamością IAM oraz wielowęzłowy klaster Kubernetes [11].

8.3 Obszary optymalizacji

Przeprowadzona implementacja wskazuje jednocześnie obszary, których rozbudowa przyniosłaby wymierne korzyści operacyjne. Najbardziej perspektywicznym kierunkiem jest rozszerzenie automatycznych testów jakości modelu poza metryki Dice i IoU o miary efektywności obliczeniowej: czas inferencji, zużycie pamięci operacyjnej oraz zachowanie modelu na danych brzegowych. Uzupełnienie bramki jakości o te kryteria pozwoliłoby wykrywać regresje nieodzwierciedlone w metrykach segmentacji. Zasadne byłoby również automatyczne generowanie raportów porównawczych przy każdym pull request, zawierających zestawienie wizualizacji predykcji obok siebie, nie tylko wartości liczbowych metryk.

Zrealizowany monitoring oparty na Prometheus i Grafanie obejmuje metryki operacyjne działającego serwisu, natomiast nie uwzględnia zjawiska dryfu danych. Wdrożenie mechanizmu analizującego rozkład statystyczny napływających danych produkcyjnych w odniesieniu do danych treningowych umożliwiłoby automatyczne sygnalizowanie momentu, w którym model wymaga ponownego uczenia na zaktualizowanym zbiorze, domykając pętlę sprzężenia zwrotnego między środowiskiem produkcyjnym a procesem uczenia. Rozważenia wymaga również zastąpienie obecnego podejścia natychmiastowej podmiany modelu mechanizmem stopniowego wdrożenia z automatycznym wycofaniem zmian w przypadku pogorszenia metryk produkcyjnych, co dodatkowo ograniczyłoby ryzyko regresji niezidentyfikowanej przez bramkę jakości.

8.4 Podsumowanie

Przeprowadzone badanie potwierdza, że zastosowanie technik DevOps i MLOps do automatyzacji procesu ponownego uczenia i wdrażania modeli AI jest technicznie wykonalne w ramach ograniczonego budżetu akademickiego oraz ekonomicznie uzasadnione już przy niewielkiej skali operacji. Automatyzacja nie tylko redukuje nakład pracy, lecz zmienia jej charakter: rola inżyniera ewoluuje od wykonawcy powtarzalnych czynności ku projektantowi procesu zarządzanego przez zautomatyzowaną infrastrukturę. Podejście manualne, choć prostsze w uruchomieniu, nie skaluje się wraz z rosnącą częstotliwością aktualizacji ani z powiększającym się zbiorem danych: ryzyko błędu kumuluje się z każdą iteracją, a czas i koszt rosną proporcjonalnie do rozmiaru danych i liczby wdrożeń. Zaprojektowana architektura, oparta wyłącznie na narzędziach open-source i standardowej infrastrukturze chmurowej, może zostać przeniesiona na inne typy modeli i zadania uczenia maszynowego przy minimalnych modyfikacjach skryptów walidacji i bramki jakości, stanowiąc empirycznie zweryfikowany wzorzec automatyzacji MLOps możliwy do zastosowania w środowisku produkcyjnym.

8.5 Wykorzystanie narzędzi sztucznej inteligencji

W trakcie realizacji pracy inżynierskiej korzystano z narzędzi opartych na modelach językowych: Claude Code (konsolowy asystent programistyczny), ChatGPT oraz Gemini. Dominującym narzędziem w tym projekcie był Claude Code, stosowany w trybie interaktywnym przez cały okres implementacji. Współpraca ta miała charakter iteracyjny i konwersacyjny: asystent proponował rozwiązanie, autor weryfikował je w działającym środowisku (AWS Academy, k3s, GitHub Actions), a wyniki testów stawały się podstawą kolejnej rundy dyskusji i poprawek. Taki cykl był szczególnie wartościowy przy diagnozowaniu trudnych do odtworzenia błędów infrastrukturalnych, takich jak konflikty pakietów systemowych na Amazon Linux 2023, nieprawidłowe śledzenie plików przez DVC czy ograniczenia uprawnień IAM specyficzne dla konta laboratoryjnego AWS Academy.

Narzędzia AI były wykorzystywane przede wszystkim do tworzenia i optymalizacji złożonych skryptów Bash i skryptów workflow GitHub Actions, pisania modułów Terraform, rozwiązywania problemów z konfiguracją k3s i Helm oraz do wsparcia przy pisaniu treści niniejszej pracy dyplomowej w LaTeX. W wielu przypadkach asystent pomagał zrozumieć przyczyny problemów na poziomie dokumentacji narzędzi i zachowania systemów, co skracало czas potrzebny na samodzielne docieranie do tych informacji. Charakter projektu łączącego jednocześnie infrastrukturę chmurową, orkiestrację kontenerów, potoki MLOps i uczenie maszynowe sprawiał, że przestrzeń problemu była zbyt rozległa, by opierać się wyłącznie na jednym źródle wiedzy.

Należy podkreślić, że proponowane przez narzędzia rozwiązania nie zawsze były poprawne od razu i wymagały weryfikacji oraz korekty. Ostateczne decyzje architektoniczne, dobór technologii, integracja poszczególnych komponentów systemu oraz analiza i interpretacja wyników pozostały w całości domeną autora pracy.

Literatura

- [1] Yevgeniy Brikman. *Terraform: Up and Running*. O'Reilly Media, 3 edition, 2022. ISBN 978-1-098-11674-3. <https://cdn.bookey.app/files/pdf/book/en/terraform.pdf>.
- [2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. In *ACM Queue*, volume 14, pages 70–93, 2016. doi: 10.1145/2898442.2898444. <https://dl.acm.org/doi/10.1145/2898442.2898444>.
- [3] Scott Chacon and Ben Straub. *Pro git*. <https://git-scm.com/book>, 2014. Dostęp: 10.02.2026.
- [4] DataFight. U-net i segmentacja obrazu. <https://datafight.wordpress.com/2020/05/04/u-net-i-segmentacja-obrazu/>, 2020. Dostęp: 10.02.2026.
- [5] MLOPS Flow Diagram. Mlops lifecycle diagram. <https://bluemetrika.com/czym-jest-mlops/>, 2024. Dostęp: 16.02.2026.
- [6] GitHub Inc. Github actions documentation. <https://docs.github.com/en/actions>, 2024. Dostęp: 10.02.2026.
- [7] Helm Authors. Helm — the kubernetes package manager. <https://helm.sh/docs/>, 2023. Dostęp: 10.02.2026.
- [8] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010. ISBN 978-0-321-60191-9. <https://www.informit.com/store/continuous-delivery-9780321601919>.
- [9] Iterative.ai. Data version control — open-source version control system for machine learning projects. <https://dvc.org/doc>, 2023. Dostęp: 10.02.2026.
- [10] Kaggle. Yandex toloka water meters dataset. <https://www.kaggle.com/datasets/tapakah68/yandextoloka-water-meters-dataset>, 2023. Dostęp: 10.02.2026.
- [11] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879, 2023. doi: 10.1109/ACCESS.2023.3262138. <https://ieeexplore.ieee.org/document/10081336> (open access).
- [12] Grafana Labs. Grafana documentation: Introduction. <https://grafana.com/docs/grafana/latest/fundamentals/>, 2024. Dostęp: 16.02.2026.
- [13] Mike Loukides. What is devops? <https://cdn.bookey.app/files/pdf/book/en/what-is-devops-.pdf>, 2012. Bookey summary PDF, accessed: 2026-02-16.
- [14] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2014. <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>.

- [15] MLflow Contributors. MLflow: An open source platform for the machine learning lifecycle. <https://mlflow.org>, 2018. Dostęp: 10.02.2026.
- [16] Nehal Navlani. Ci/cd pipeline - system design. <https://www.geeksforgeeks.org/system-design/cicd-pipeline-system-design/>, 2025.
- [17] MLflow Project. Mlflow model registry. <https://mlflow.org/docs/latest/ml/model-registry/>, 2024. Dostęp: 16.02.2026.
- [18] Prometheus Authors. Prometheus — monitoring system and time series database. <https://prometheus.io/docs/introduction/overview/>, 2023. Dostęp: 10.02.2026.
- [19] Rancher Labs. k3s — lightweight kubernetes. <https://docs.k3s.io>, 2023. Dostęp: 10.02.2026.